Design, Implementation, Verification, and Performance Evaluation of a PDSCH-Oriented 5G NR Massive MIMO-OFDM Physical Layer Link-Level Simulator

5G Physical-Layer Simulation, Algorithm Design, and Engineering Verification Portfolio

Topics: PDSCH Processing · LDPC Coding & Rate Matching · CP-OFDM · Massive MIMO Precoding · DM-RS Channel Estimation (LS/MMSE) · ZF/MMSE Equalization · AWGN/Rayleigh/Rician/3GPP TDL Channels · Monte Carlo BER/BLER/EVM/Throughput Analysis

Tools: MATLAB (primary simulation and verification platform) · Python (NumPy, SciPy, Matplotlib)

Author: Md Moklesur Rahman | moklesur.eee@gmail.com | linkedin.com/in/md-moklesur-rahman-65a63962 | github.com/dipucwc

Chapter 1. Introduction

1.1 Background and Motivation

Mobile data traffic grows every year. More devices connect to the network, and users expect fast and reliable wireless service. To meet these needs, the Third Generation Partnership Project (3GPP) developed Fifth Generation New Radio (5G NR) [1]. 5G NR is a flexible radio technology. It supports three main service types: enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communications (URLLC), and massive Machine-Type Communications (mMTC).

Compared with older cellular systems, 5G NR adds several new features. These include scalable numerology, modern channel coding, high-order modulation, flexible frame structures, Massive Multiple-Input Multiple-Output (Massive MIMO), digital beamforming, and Orthogonal Frequency Division Multiplexing (OFDM) [2], [3]. Together, these features improve spectral efficiency, network capacity, reliability, and user data rates.

Two of these technologies form the core of the NR physical layer: OFDM and Massive MIMO. OFDM splits a wide frequency band into many narrow subcarriers. Each subcarrier sees an almost flat channel, so equalization at the receiver becomes simple. This makes OFDM strong against multipath propagation. Massive MIMO uses a large number of antennas at the base station. More antennas give higher beamforming gain, better spatial diversity, and the ability

to send several data streams at the same time. The result is higher capacity and better spectral efficiency.

The Physical Downlink Shared Channel (PDSCH) is the main downlink data channel in 5G NR. It carries user data from the base station to the user equipment [2], [4]. PDSCH performance depends on a long chain of processing steps: transport-block processing, channel coding, modulation, resource mapping, OFDM waveform generation, the wireless channel itself, channel estimation, equalization, demodulation, and channel decoding. These steps are strongly connected. An error in one step affects all the steps after it. For this reason, we cannot judge the performance of one algorithm alone. We must test the full chain together.

This is why link-level simulation is the standard method in physical-layer engineering. A link-level simulator is a software model of the complete link: transmitter, channel, and receiver. It lets us test algorithms under controlled and repeatable conditions before any hardware is built. Compared with hardware prototyping, simulation is cheaper, faster, easier to debug, and fully reproducible. At the same time, it stays close to the theoretical models of communication.

The goal of this project is to develop a 3GPP 5G NR PDSCH-Based Massive MIMO-OFDM Physical Layer Link-Level Simulator. The simulator models the main digital baseband processing chain of the NR downlink. It is built to support wireless research, algorithm development, engineering verification, and performance evaluation, using standard 3GPP channel models and reproducible Monte Carlo simulations.

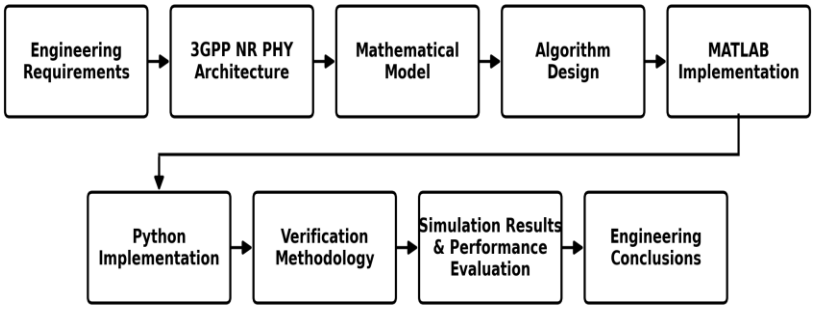


Figure 1. Overall development workflow of the proposed 3GPP 5G NR PDSCH-based Massive MIMO-OFDM physical layer link-level simulator.

Figure 1 illustrates the overall workflow of the project. The work starts with the engineering requirements and the 3GPP NR physical-layer architecture. From these, we build the mathematical models and design the algorithms. The algorithms are then implemented twice: once in MATLAB and once in Python. Both implementations are checked with a clear verification methodology. After verification, we run full link-level simulations and evaluate

the performance. The project ends with the engineering conclusions. This workflow gives complete traceability: every result can be traced back through the code, the algorithms, the equations, and the requirements.

1.2 Problem Statement

A 5G NR PDSCH link contains many connected transmitter, channel, and receiver blocks. Simplified simulators often omit important functions such as LDPC coding, DM-RS channel estimation, MIMO processing, and realistic 3GPP channels. Therefore, a complete and configurable link-level simulator is needed for accurate performance analysis and algorithm comparison.

1.3 Objectives and Scope

The simulator follows the PDSCH processing of TS 38.211 [2], TS 38.212 [3], and TS 38.214 [4]. The transmitter covers transport-block processing, CRC attachment, LDPC coding, rate matching, scrambling, modulation from QPSK to 256-QAM, layer mapping, digital precoding with SVD eigenbeamforming and maximum-ratio transmission, resource mapping with DM-RS insertion, and CP-OFDM modulation. The receiver covers cyclic-prefix removal, FFT processing, resource demapping, DM-RS-based least-squares (LS) and minimum mean square error (MMSE) channel estimation, Zero-Forcing (ZF) and MMSE equalization, soft demodulation, descrambling, rate recovery, LDPC decoding, and CRC checking, with ideal timing and frequency synchronization assumed so that estimation and detection can be studied without synchronization errors.

The scope is the digital baseband of a single-cell, single-user downlink. Antenna configurations reach 64×8 with up to 4 spatial layers, the channels are AWGN, Rayleigh, Rician, and 3GPP TDL [5], and the metrics are BER, BLER, EVM, throughput, spectral efficiency, post-equalization SINR, channel-estimation NMSE, and MIMO capacity, all evaluated by seeded Monte Carlo simulation. Higher-layer functions, synchronization algorithms, and RF impairments such as phase noise, IQ imbalance, and PA nonlinearity are excluded and identified as future extensions. Every result figure in this report is generated from executed simulator code and its saved numerical files, under the provenance and acceptance rules of Chapters 10 and 11.

1.4 Engineering Contributions

Three contributions define the project. The first is a modular link-level simulator that combines LDPC coding, Massive MIMO transmission, OFDM, DM-RS-based estimation, standard channel models, and linear equalization in one architecture with block-replaceable interfaces. The second is the pair of independently written MATLAB and Python implementations, cross-verified against each other with a recorded, tolerance-based comparison. The third is the verification methodology itself: results are accepted only after

matching closed-form references, including theoretical BER in AWGN and Rayleigh fading and ergodic MIMO capacity, so correctness is demonstrated quantitatively rather than assumed.

1.5 Organization of the Report

Chapter 2 defines the requirements and system specification. Chapter 3 presents the physical-layer architecture, Chapter 4 the mathematical foundations, Chapter 5 the end-to-end model, Chapter 6 the channel models, and Chapter 7 the signal-processing algorithms. Chapters 8 and 9 describe the MATLAB and Python implementations, Chapter 10 the verification methodology, and Chapter 11 the executed simulation results of both implementations, including their cross-verification record. Chapter 12 condenses the engineering trade-offs and Chapter 13 concludes with the contributions, the remaining open item, and the future work.

Chapter 2. System Requirements and Specification

This Sections defines the engineering requirements, the system architecture, the functional specification, the simulation assumptions, and the performance metrics that everything later builds on. The simulator is specified as a modular physical-layer software platform: every processing block is an independent module with a clean interface, so the transmitter, the channel, the receiver, and the metric functions can be changed or extended without redesigning the complete simulator.

2.1 System Requirements

The simulator shall model the main digital baseband chain of a 3GPP 5G NR downlink PDSCH link per TS 38.211 [2], TS 38.212 [3], and TS 38.214 [4], from transport-block generation to recovery, with ideal synchronization assumed at the receiver. It shall support QPSK through 256-QAM, antenna configurations from 2×2 up to 64×8 with up to 4 spatial layers, the AWGN, Rayleigh, Rician, and 3GPP TDL channels [5], and reproducible Monte Carlo evaluation over configurable SNR points. Table 1 states the ten functional requirements with the verification method of each one, which makes the set testable and lets Chapter 10 trace every test back to a numbered requirement.

ID	Requirement (the simulator shall …)	Verification Method
R1	Implement the full PDSCH transmit chain: CRC, LDPC coding, rate matching, scrambling, modulation, layer mapping, precoding, resource mapping, DM-RS insertion, and CP-OFDM modulation.	Unit test of each block; end-to-end loopback with an ideal channel.
R2	Implement the full receive chain: CP removal, FFT, resource demapping, LS and MMSE channel estimation, ZF and MMSE equalization, soft demodulation, rate recovery, LDPC decoding, and CRC check.	Loopback test: zero errors at high SNR in AWGN.
R3	Support QPSK, 16-QAM, 64-QAM, and 256-QAM modulation with correct constellation mapping.	Constellation inspection and EVM check against reference symbols.
R4	Support antenna configurations from 2×2 up to 64×8 with up to 4 spatial layers.	Configuration sweep; dimension checks on the resource grid and channel matrices.
R5	Support AWGN, Rayleigh, Rician, and 3GPP TDL channel models.	Statistical validation of fading distributions and power-delay profiles.
R6	Use a baseline numerology of 30 kHz SCS, 100 MHz bandwidth, and 273 resource blocks in FR1 [6], with configurable parameters.	Grid dimension check against 3GPP values.
R7	Compute BER, BLER, EVM, throughput, spectral efficiency, SINR, NMSE, and capacity.	Comparison with closed-form theory where available.
R8	Run reproducible Monte Carlo simulations with random-seed control.	Repeated runs with the same seed give identical results.
R9	Provide a MATLAB implementation and an independent Python implementation with the same architecture.	Cross-language comparison of key results.
R10	Use a modular architecture in which any processing block can be replaced without changing the others.	Interface review; block-swap regression test.

Table 1. Functional requirements of the simulator and their verification methods.

2.2 System Architecture Specification

The simulator consists of four main subsystems: the NR PDSCH transmitter, the wireless propagation channel, the NR PDSCH receiver, and the performance evaluation module. The transmitter converts a transport block into a multi-antenna OFDM waveform. The channel module applies fading, propagation effects, and receiver noise. The receiver estimates the channel, equalizes the MIMO signal, demodulates and decodes the received data, and verifies transport-block recovery using the CRC. The performance evaluation module computes the link-level metrics and generates the simulation figures.

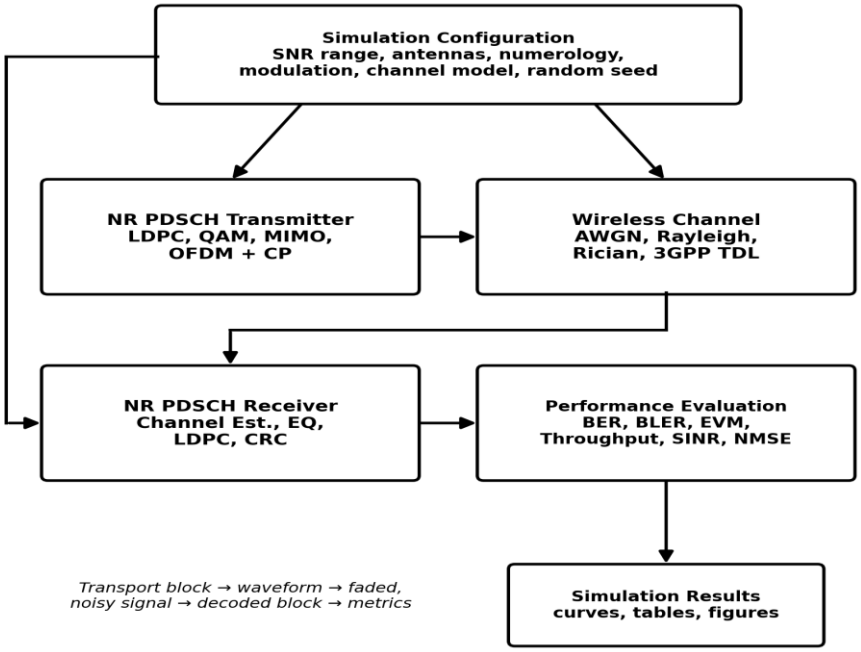


Figure 2. High-level system architecture of the simulator.

The high-level system architecture of the simulator is shown in Figure 2. The simulation configuration block, at the top, controls the numerology, the modulation and coding scheme, the antenna configuration, the channel model, the SNR points, and the random seeds, so every simulation run is consistent and reproducible. The transport block enters the NR PDSCH transmitter and leaves it as a multi-antenna OFDM waveform. The wireless channel block applies the selected fading model and adds receiver noise. The NR PDSCH receiver recovers the transport block, and the performance evaluation block turns the accumulated statistics into the link-level metrics: BER, BLER, EVM, throughput, spectral efficiency, SINR, NMSE, and capacity. The simulation results block collects the final outputs of every run: the performance curves, the tables, and the figures used in this report.

2.3 Functional Specification

The transmitter module generates transport blocks, performs channel coding, maps the coded bits onto modulation symbols, applies digital precoding, builds the NR resource grid, multiplexes the DM-RS symbols, and generates the time-domain OFDM waveform. The channel module applies the selected propagation model and produces the received signal by combining the fading effects with additive receiver noise. The receiver module estimates the channel from the DM-RS symbols, applies ZF or MMSE equalization, recovers the modulation symbols, decodes the received transport block, and checks decoding success with the CRC. The performance evaluation module repeats the complete transmission and reception process over multiple SNR values and channel realizations. It accumulates the communication statistics and generates the BER, BLER, throughput, spectral efficiency, EVM, SINR, NMSE, and capacity results.

2.4 Simulation Assumptions and Baseline Parameters

The baseline is a single-cell, single-user downlink PDSCH in the digital baseband domain with ideal synchronization and an ideal RF front end: phase noise, IQ imbalance, PA nonlinearity, oscillator drift, and quantization are excluded from the first implementation stage, so the baseband algorithms can be studied on their own. The channel is quasi-static over one slot unless Doppler is enabled, noise is additive white Gaussian, and Monte Carlo trials use independent blocks and realizations under random-seed control, so every result reproduces. Table 2 lists the baseline simulation parameters.

Parameter	**Baseline Value**
Frequency range	FR1
Subcarrier spacing	30 kHz
Channel bandwidth	100 MHz (273 resource blocks) [6]
FFT size	4096
Cyclic prefix	Normal
Downlink channel	PDSCH
Channel coding	LDPC with rate matching
Modulation	QPSK, 16-QAM, 64-QAM, 256-QAM
Antenna configuration	2×2 up to 64×8, up to 4 spatial layers
Reference signal	DM-RS
Channel models	AWGN, Rayleigh, Rician, 3GPP TDL
Synchronization	Ideal timing and frequency
Simulation method	Monte Carlo with random-seed control

Table 2. Baseline simulation parameters of the simulator.

2.5 Performance Metrics

The simulator evaluates physical-layer performance using BER, BLER, throughput, spectral efficiency, EVM, post-equalization SINR, channel-estimation NMSE, and MIMO channel capacity. BER measures bit-level detection reliability. BLER measures transport-block decoding success after the CRC check. Throughput and spectral efficiency quantify the useful information delivered as a function of the SNR, the modulation order, the coding rate, the antenna configuration, and the channel condition.

EVM measures modulation accuracy after equalization. Post-equalization SINR characterizes the quality of the receiver output per layer or per resource element. NMSE measures the accuracy of the DM-RS-based channel estimation. MIMO channel capacity provides a theoretical benchmark for the spatial multiplexing potential of the selected antenna configuration and channel realization [7].

Chapter 3. 3GPP 5G NR Physical Layer Architecture

This Sections defines the physical-layer architecture of the simulator within its downlink PDSCH link-level scope. The implemented chain follows the main NR downlink procedures of TS 38.211 [2], TS 38.212 [3], and TS 38.214 [4]; the simulator does not claim a conformance implementation of every optional NR feature, but implements the PDSCH-focused baseband functions needed for link-level algorithm evaluation [11]. The four subsystems introduced in Section 2.2 and Figure 2 are opened one by one: the transmitter chain, the resource grid, the Massive MIMO processing model, the wireless channel, the receiver chain, and the Monte Carlo simulation flow.

3.1 NR PDSCH Transmitter Architecture

The NR PDSCH transmitter converts an input transport block into a multi-antenna time-domain OFDM waveform. The processing begins with transport-block CRC attachment. If the transport block is larger than the applicable code-block size, code-block segmentation is applied, and each code block receives its own CRC. Each code block is then LDPC encoded, rate matched, and concatenated. The concatenated bit stream is scrambled. The scrambled bits are mapped to QPSK, 16-QAM, 64-QAM, or 256-QAM symbols, according to the selected modulation order.

The modulation symbols are mapped to one or more spatial layers. Digital precoding transforms the layer-domain symbols into antenna-port signals. The precoded symbols are mapped onto the allocated PDSCH resource elements, and DM-RS symbols are multiplexed into their configured resource elements to support channel estimation at the receiver. The populated resource grid is then converted into a time-domain waveform using IFFT-based OFDM modulation, followed by cyclic-prefix insertion.

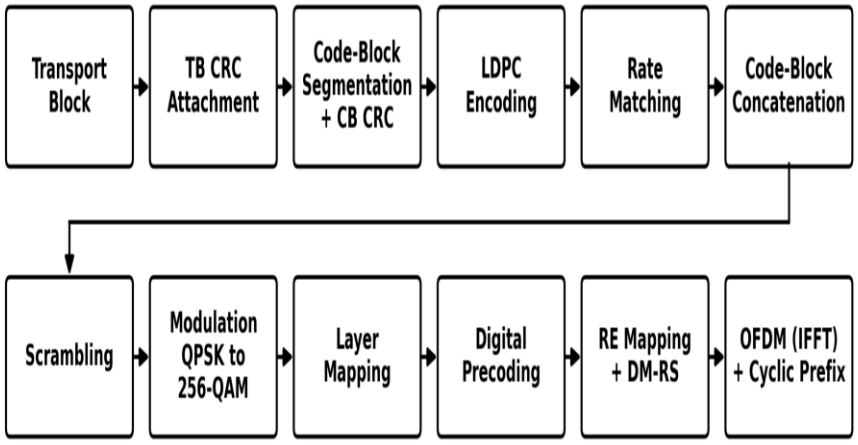


Figure 3. Implemented NR PDSCH transmitter processing chain.

Figure 3 presents the transmit path from the input transport block to the multi-antenna OFDM waveform. The upper row covers the bit-level processing: CRC attachment, code-block

segmentation with code-block CRC, LDPC encoding, rate matching, and concatenation. The lower row covers the symbol-level processing: scrambling, modulation, layer mapping, digital precoding, resource-element mapping with DM-RS insertion, and OFDM modulation with cyclic-prefix insertion. The order of the blocks follows the processing structure of TS 38.211 and TS 38.212.

3.2 NR Resource Grid

The NR resource grid is the frequency-domain representation of the transmitted signal before OFDM modulation. It is organized over subcarriers in frequency and OFDM symbols in time. Each time-frequency location is one resource element, and twelve consecutive subcarriers form one physical resource block [2].

In this simulator, the PDSCH symbols occupy the allocated data resource elements, and the DM-RS symbols occupy the configured pilot positions used for channel estimation. The resource grid is the interface between the coded modulation processing and the OFDM waveform generation. Correct resource mapping is essential, because the channel estimation, the equalization, the demodulation, and the EVM, throughput, and BLER calculations all depend on the same time-frequency allocation.

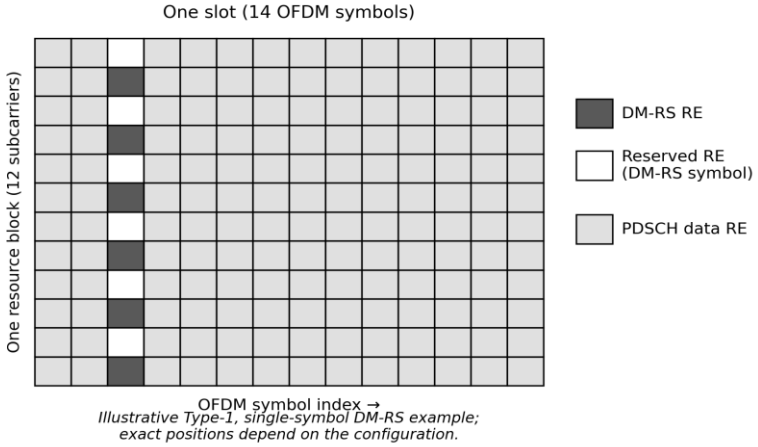


Figure 4. Simplified NR resource grid with PDSCH and DM-RS mapping.

One resource block over one slot is shown in Figure 4: twelve subcarriers in frequency and fourteen OFDM symbols in time. The dark squares mark the DM-RS resource elements, placed on alternating subcarriers of one OFDM symbol in this illustrative Type-1, single-symbol example. The white squares in the same symbol are reserved, and the light squares carry the PDSCH data. The exact DM-RS positions in the simulator depend on the selected DM-RS configuration and the resource allocation.

3.3 Massive MIMO Transmission Architecture

Massive MIMO processing maps the spatial layers to the transmit antenna ports using a digital precoding matrix. Let L be the number of transmission layers, NT the number of transmit antennas, and NR the number of receive antennas, with L never larger than the smaller of NT and NR. For a layer vector s of size L by 1, the precoding matrix W of size NT by L generates the antenna-domain transmit vector

$$x = Ws \tag{1}$$

The received vector of size NR by 1 is modeled as

$$y = Hx + n = HWs + n \tag{2}$$

where H is the NR by NT MIMO channel matrix and n is the NR by 1 additive receiver noise vector. The product HW acts as the effective channel seen by the layers. The receiver applies ZF or MMSE equalization to this model to separate the transmitted spatial layers; the equalizer derivations are given in Chapter 7.

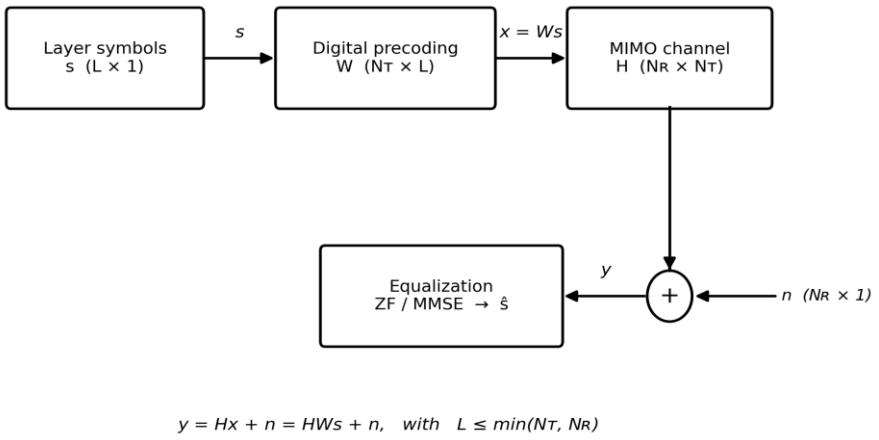


Figure 5. Massive MIMO layer mapping, precoding, channel, and equalization model.

Figure 5 illustrates the signal path of equation (2). The layer vector s enters the digital precoder W, which produces the antenna-domain vector x with one entry per transmit antenna. The MIMO channel H mixes the transmit antennas onto the receive antennas, and the noise vector n is added at the receiver input. The equalizer, ZF or MMSE, processes the received vector y and produces the layer estimates. The dimension constraint under the diagram states that the number of layers can never exceed the smaller antenna count.

3.4 Wireless Channel Architecture

The wireless channel module models the propagation between the transmit and receive antenna arrays. The AWGN model is the baseline case and is used for analytical verification. Fading evaluation uses Rayleigh fading, Rician fading, and selected 3GPP TDL channel

models [5]. In a link-level simulator, the large-scale effects, path loss and shadowing, are not modeled as separate blocks; they are absorbed into the definition of the operating SNR. The small-scale fading is modeled explicitly. The channel is quasi-static over one slot, and Doppler can be enabled when mobility effects are evaluated, consistent with the assumptions of Section 2.4.

The received waveform is generated by applying the selected MIMO channel response to the transmitted waveform and adding receiver noise. In frequency-selective channels, different subcarriers see different channel responses. This directly affects the channel estimation accuracy, the equalization performance, and through them the EVM, BER, BLER, throughput, and spectral efficiency.

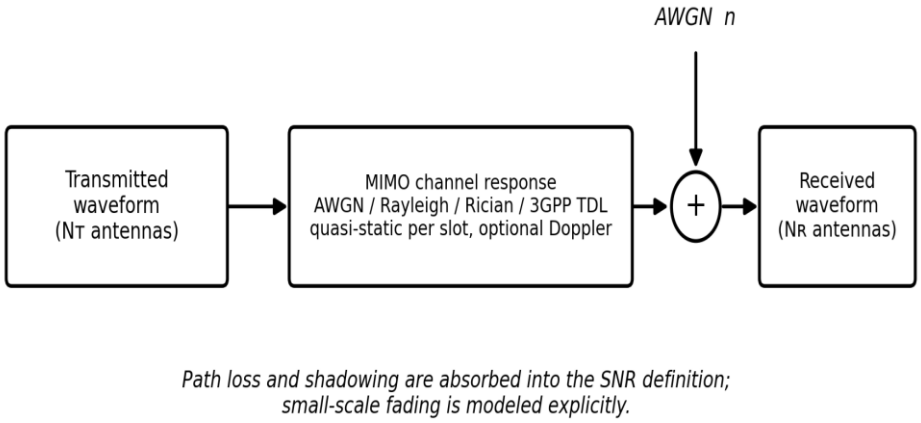


Figure 6. Wireless channel processing model.

The path of the transmitted waveform through the channel module appears in Figure 6. The multi-antenna waveform enters the MIMO channel response block, which applies the selected model: AWGN, Rayleigh, Rician, or a 3GPP TDL profile, quasi-static over one slot with optional Doppler. Additive white Gaussian noise is added at the output, producing the received waveform on the receive antennas. The note under the diagram records the link-level convention: path loss and shadowing are absorbed into the SNR definition, and only the small-scale fading is modeled explicitly.

3.5 NR PDSCH Receiver Architecture

The receiver performs the inverse processing needed to recover the transmitted transport block. Under the ideal-synchronization assumption of Section 2.4, the OFDM symbol boundary is known, so the cyclic prefix is removed at the known boundary and FFT processing reconstructs the received resource grid. The PDSCH and DM-RS resource elements are then separated.

The extracted DM-RS symbols are used to estimate the channel frequency response, using the LS and MMSE estimators. The estimated channel is supplied to the MIMO equalizer, where the ZF and MMSE algorithms are evaluated under identical channel and noise

realizations, so that their performance can be compared fairly. After equalization, soft demodulation produces bit reliability information. The receiver then performs descrambling, rate recovery, LDPC decoding, and CRC verification.

The CRC check decides transport-block success or failure and drives the BLER statistic. HARQ retransmission is outside the scope of this PHY-only link-level simulator, as stated in Chapter 2.

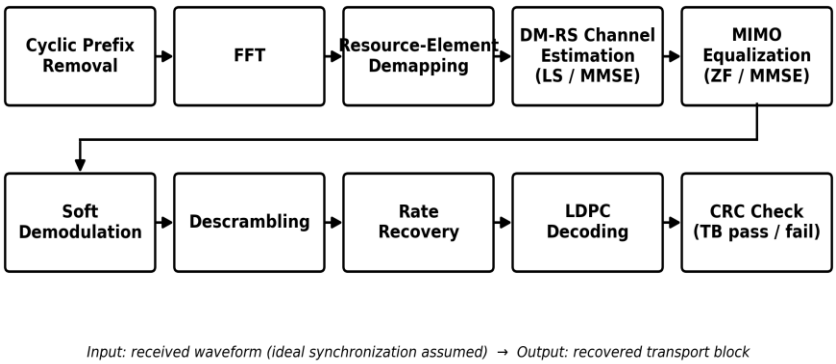


Figure 7. Implemented NR PDSCH receiver processing chain.

Figure 7 presents the receive path from the received waveform to the recovered transport block. The upper row covers the waveform and grid processing: cyclic-prefix removal at the known symbol boundary, FFT, resource-element demapping, DM-RS-based channel estimation with the LS and MMSE estimators, and MIMO equalization with the ZF and MMSE algorithms. The lower row covers the bit-level recovery: soft demodulation, descrambling, rate recovery, LDPC decoding, and the final CRC check that declares the transport block passed or failed.

3.6 Monte Carlo Link-Level Simulation Flow

The simulator evaluates physical-layer performance using Monte Carlo simulation. For each SNR point, independent transport blocks and channel realizations are generated. The transmitter produces the PDSCH waveform, the channel applies the propagation effects and the receiver noise, and the receiver attempts to recover the transport block. Random-seed control keeps every experiment reproducible while the trials remain statistically independent.

The resulting bit errors, block errors, EVM, SINR, channel-estimation NMSE, throughput, spectral efficiency, and MIMO capacity values are accumulated and averaged over the realizations.

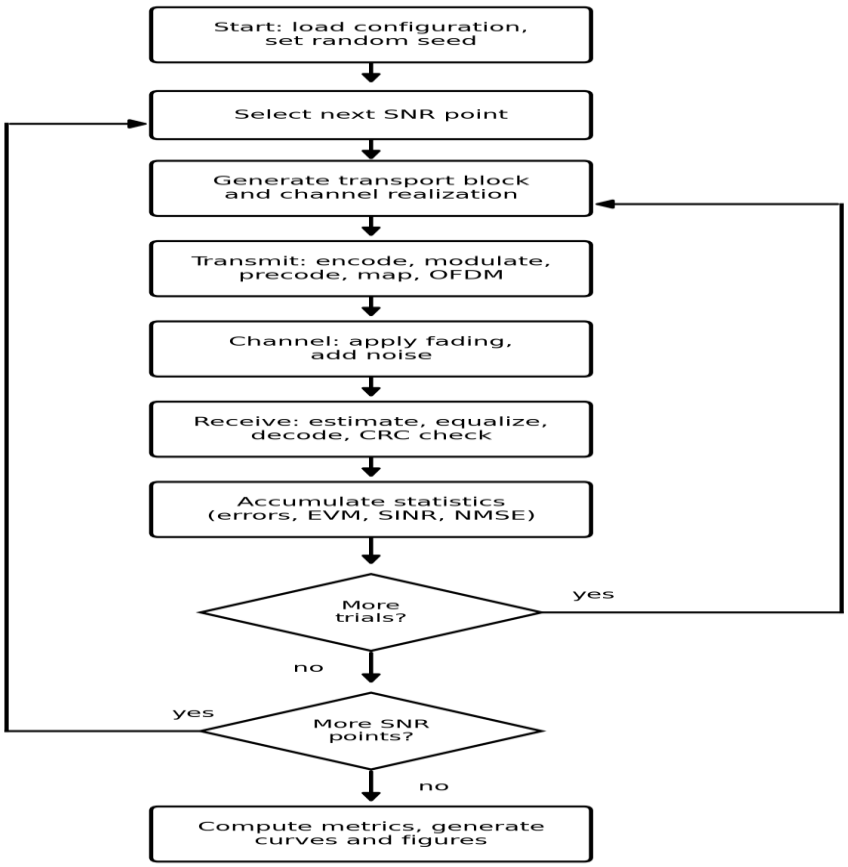


Figure 8. Monte Carlo simulation flow for link-level performance evaluation.

The two nested loops of the simulation are shown in Figure 8. The inner loop repeats the transmit, channel, receive, and accumulate steps for independent transport blocks and channel realizations at one SNR point. When enough trials have been collected, the outer loop moves to the next SNR point and repeats the inner loop. When all SNR points are finished, the accumulated statistics are turned into the final metrics, curves, and figures. The random seed is set once at the start, so the complete experiment can be reproduced exactly.

Chapter 4. Mathematical Foundations

This Sections presents the mathematical foundations used by the 3GPP 5G NR PDSCH-Based Massive MIMO-OFDM Physical Layer Link-Level Simulator. The simulator works in the complex baseband domain. The modulation symbols, the OFDM samples, the channel coefficients, the noise samples, and the MIMO vectors are all complex-valued discrete-time signals. This modeling choice keeps all the physical-layer behavior needed for link-level evaluation, and it avoids unnecessary RF carrier-level modeling [8]. The foundations given here are used in the complete system model of Chapter 5 and in the algorithm derivations of Chapter 7.

4.1 Complex Baseband Signal Representation

A passband RF signal can be represented by its equivalent complex baseband signal. If s(t) is the complex baseband signal and fc is the carrier frequency, the real passband signal is

$$x_{RF}(t) = Re\{ s(t)\, e^{j2\pi f_c t} \} \tag{3}$$

The complex baseband signal has an in-phase part and a quadrature part:

$$s(t) = s_I(t) + j\, s_Q(t) \tag{4}$$

In discrete-time simulation, the transmitted baseband sequence is written as

$$s[n] = s_I[n] + j\, s_Q[n] \tag{5}$$

where n is the discrete-time sample index. All processing in the simulator uses this discrete-time complex baseband form.

4.2 OFDM Signal Model

Let X[k] be the complex modulation symbol on subcarrier k, with k = 0, 1, ..., N - 1, where N is the FFT size. The time-domain OFDM sample is produced by the inverse discrete Fourier transform:

$$x[n] = (1/\sqrt{N}) \sum_{k=0}^{N-1} X[k]\, e^{j2\pi kn/N}, \quad n = 0, 1, ..., N - 1 \tag{6}$$

At the receiver, after cyclic-prefix removal, the FFT reconstructs the received frequency-domain subcarriers:

$$Y[k] = (1/\sqrt{N}) \sum_{n=0}^{N-1} y[n]\, e^{-j2\pi kn/N} \tag{7}$$

The derivation uses the unitary DFT convention: both the IFFT and the FFT carry the same normalization factor $1/\sqrt{N}$. With this convention, the transform preserves signal power. The MATLAB and Python implementations apply the same normalization explicitly, so the numerical simulations stay consistent with the mathematical model.

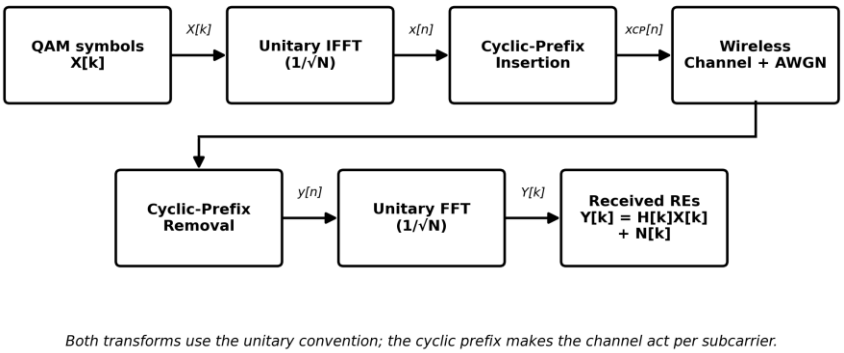


Figure 9. OFDM baseband processing model used in the simulator.

Figure 9 illustrates the path of the frequency-domain QAM symbols through the OFDM chain. The unitary IFFT turns the symbols X[k] into time-domain samples, the cyclic prefix is inserted, and the waveform passes through the wireless channel where noise is added. At the receiver, the cyclic prefix is removed and the unitary FFT recovers the received resource elements Y[k]. When the cyclic prefix is long enough, each subcarrier obeys the simple per-subcarrier model Y[k] = H[k]X[k] + N[k] given in equation (11).

4.3 Subcarrier Orthogonality

The orthogonality of the OFDM subcarriers follows from the discrete Fourier basis. For two subcarriers k and m, the inner product over one OFDM symbol is

$$\sum_{n=0}^{N-1} e^{j2\pi kn/N} e^{-j2\pi mn/N} = \sum_{n=0}^{N-1} e^{j2\pi(k-m)n/N} \quad (8)$$

This geometric sum has a simple closed form:

$$\sum_{n=0}^{N-1} e^{j2\pi(k-m)n/N} = N \text{ for } k = m, \text{ and } 0 \text{ for } k \neq m \quad (9)$$

Because of this property, subcarriers that overlap in frequency can still be separated perfectly by FFT processing, as long as synchronization is ideal and the channel does not break the orthogonality. This is the mathematical reason why the OFDM receiver can work subcarrier by subcarrier.

4.4 Cyclic Prefix

In a multipath channel, the received signal is the linear convolution of the transmitted waveform and the channel impulse response. The cyclic prefix turns this linear convolution into a circular convolution, provided that the cyclic-prefix length is greater than or equal to the maximum effective delay spread of the channel. If the useful OFDM symbol is x[0], x[1], ..., x[N - 1] and the cyclic-prefix length is NCP, the transmitted symbol with cyclic prefix is

$$x_{CP}[n] = x[(n - N_{CP}) \bmod N], \quad n = 0, 1, ..., N + N_{CP} - 1 \quad (10)$$

With the cyclic prefix in place, and with the channel quasi-static over one slot as assumed in Section 2.4, the circular convolution becomes a simple multiplication in the frequency domain. After cyclic-prefix removal and FFT processing, each received subcarrier obeys

$$Y[k] = H[k]\,X[k] + N[k] \tag{11}$$

where H[k] is the channel frequency response on subcarrier k and N[k] is the frequency-domain noise sample. Equation (11) is the single most important result of this chapter: it reduces the frequency-selective wideband channel to one flat complex gain per subcarrier, and every estimation and equalization algorithm in the later chapters builds on it.

4.5 QAM Modulation Model

The simulator supports QPSK, 16-QAM, 64-QAM, and 256-QAM. For modulation order M, each modulation symbol carries

$$Q_m = \log_2(M) \tag{12}$$

coded bits. A QAM symbol is a complex number

$$a = a_I + j\,a_Q \tag{13}$$

where the in-phase level aI and the quadrature level aQ are taken from the constellation of the selected order [8]. The constellation is normalized so that the average symbol power is:

$$E\{\,|a|^2\,\} = 1 \tag{14}$$

This normalization keeps the SNR comparable across modulation orders: a 256-QAM curve and a QPSK curve are plotted against the same SNR axis without any hidden energy difference.

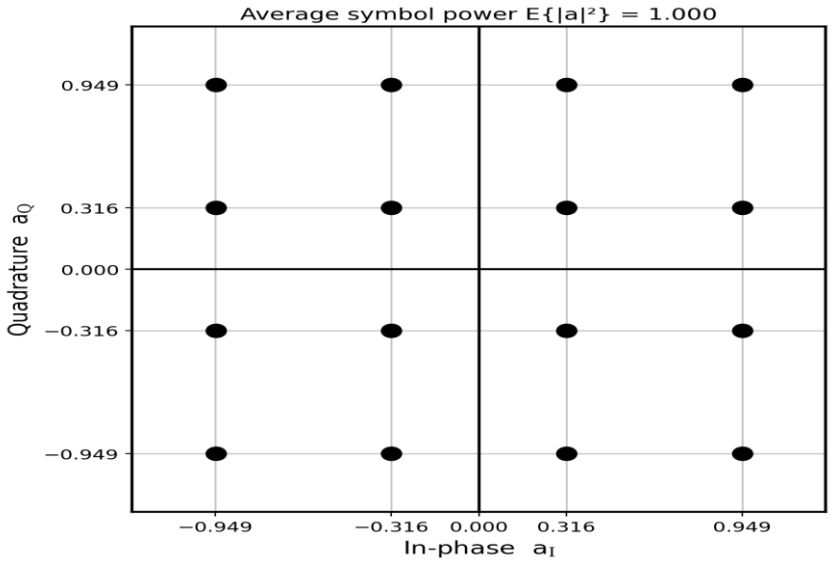


Figure 10. Normalized 16-QAM constellation.

The sixteen constellation points of 16-QAM after average-power normalization are shown in Figure 10. The in-phase and quadrature levels are the values $\pm 1/\sqrt{10}$ and $\pm 3/\sqrt{10}$, so the average symbol power over all sixteen points is exactly one, as required by equation (14). The same normalization rule is applied to QPSK, 64-QAM, and 256-QAM.

4.6 MIMO Baseband Signal Model

For a MIMO system with NT transmit antennas and NR receive antennas, the subcarrier-level baseband model extends equation (11) from scalars to vectors and matrices [10]. On subcarrier k,

$$y[k] = H[k]\, x[k] + n[k] \tag{15}$$

where x[k] is the NT by 1 transmit vector, y[k] is the NR by 1 receive vector, H[k] is the NR by NT channel matrix, and n[k] is the NR by 1 noise vector. If s[k] is the L by 1 layer vector and W[k] is the NT by L precoding matrix, then

$$x[k] = W[k]\, s[k] \tag{16}$$

and the received signal becomes

$$y[k] = H[k]\, W[k]\, s[k] + n[k] \tag{17}$$

This is the per-subcarrier form of the model introduced in equations (1) and (2) and shown in Figure 5. The same notation is used consistently in the channel estimation, the equalization, the capacity calculation, and the performance evaluation.

4.7 Noise Model

The receiver noise is modeled as complex additive white Gaussian noise. On each receive antenna, one noise sample is

$$n = n_I + j\, n_Q \tag{18}$$

where the real and imaginary parts are independent Gaussian random variables:

$$n_I \sim N(0, \sigma_n^2/2), \quad n_Q \sim N(0, \sigma_n^2/2) \tag{19}$$

so the complex sample follows a circularly symmetric complex Gaussian distribution:

$$n \sim CN(0, \sigma_n^2) \tag{20}$$

With the average signal power normalized to one by equation (14) and the unitary transforms of Section 4.2, the noise variance for a target SNR in decibels is simply

$$\sigma_n^2 = 10^{-SNR_{dB}/10} \tag{21}$$

This normalization is applied consistently after modulation, precoding, and OFDM scaling, so every SNR-dependent result in Chapter 11 has a clear and correct meaning.

4.8 Linear Equalization Basis

On each subcarrier, the receiver estimates the transmitted layer vector with a linear equalizer. The effective channel seen by the layers is

$$G[k] = H[k]\, W[k] \tag{22}$$

so the received vector can be written directly in terms of the layers:

$$y[k] = G[k]\, s[k] + n[k] \tag{23}$$

A linear detector forms the layer estimate by multiplying the received vector with an equalization matrix F[k]:

$$\hat{s}[k] = F[k]\, y[k] \tag{24}$$

The ZF equalizer and the MMSE equalizer are two specific choices of F[k]. Their derivations, together with the LS and MMSE channel estimators that provide the estimate of G[k], are developed in Chapter 7.

Chapter 5. End-to-End System Mathematical Model

This Sections develops the complete mathematical model of the digital baseband signal path used by the simulator. A transport block is processed by the 5G NR PDSCH transmitter, mapped to OFDM resource elements, transmitted through a configurable MIMO wireless channel, and recovered by the receiver using DM-RS-based channel estimation, linear equalization, soft demodulation, LDPC decoding, and CRC verification. The model is defined at link level, in line with the scope of Chapter 2, and it extends the per-subcarrier notation of Chapter 4 with the OFDM-symbol index m, so every quantity is written per subcarrier k and per OFDM symbol m.

5.1 Baseband Link-Level System Model

For one Monte Carlo realization, the transmitter input is a binary transport block of A information bits:

$$b = [b_0, b_1, ..., b_{A-1}], \quad b_i \in \{0, 1\} \tag{25}$$

After CRC attachment, LDPC coding, rate matching, scrambling, and modulation mapping, the bits become complex QAM symbols. The symbols are mapped to spatial layers and precoded to the transmit antennas. For OFDM symbol m and subcarrier k, the antenna-domain transmit vector is

$$x[k,m] = W[k,m]\, s[k,m] \tag{26}$$

where s[k,m] is the L by 1 layer vector, W[k,m] is the NT by L digital precoding matrix, L is the number of transmission layers, and NT is the number of transmit antennas. This is the model of equations (15) and (16), now carrying both indices. The received MIMO-OFDM signal is

$$y[k,m] = H[k,m]\, x[k,m] + n[k,m] = H[k,m]\, W[k,m]\, s[k,m] + n[k,m] \tag{27}$$

The product H[k,m]W[k,m] forms the effective layer channel seen by the receiver. This effective-channel view matters for PDSCH demodulation: because the DM-RS symbols are precoded together with the data, the receiver estimates the effective channel of the transmitted layers directly, and it never needs to recover the full physical antenna channel [2].

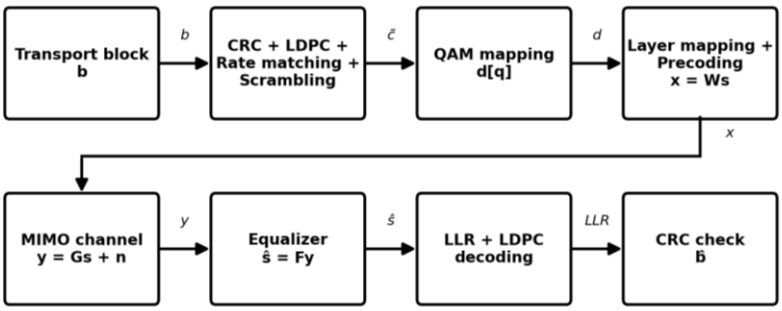


Figure 11. End-to-end 5G NR PDSCH link-level mathematical model.

The complete signal path of one Monte Carlo realization is shown in Figure 11. The transport block b is coded, rate matched, and scrambled into the bit stream, the bits are mapped to QAM symbols d[q], the symbols are arranged into layers and precoded into the antenna vector x = Ws, and the MIMO channel produces the received vector y = Gs + n. On the receive side, the equalizer forms the layer estimate with F, the soft demapper produces the LLR values, and the LDPC decoder and the CRC check deliver the decoded transport block. Every block in the figure corresponds to one equation of this chapter.

Table 3 lists the main variables of the end-to-end model, together with their meanings and dimensions. The same symbols are used in the MATLAB and Python implementations, so the code can be read next to the equations.

Symbol	Meaning	Dimension
b	Transport block (information bits)	$A \times 1$
$\tilde{c}$	Coded and scrambled bit sequence	$E \times 1$
d[q]	QAM modulation symbol	scalar, $E\{\|d\|^2\} = 1$
s[k,m]	Layer-domain symbol vector	$L \times 1$
W[k,m]	Digital precoding matrix	$N_T \times L$
x[k,m]	Antenna-domain transmit vector	$N_T \times 1$
H[k,m]	MIMO channel matrix	$N_R \times N_T$
G[k,m] = H W	Effective layer channel	$N_R \times L$
n[k,m]	Receiver noise, $CN(0, \sigma n^2 I)$	$N_R \times 1$
y[k,m]	Received vector	$N_R \times 1$
$\hat{G}$[k,m]	Estimated effective channel	$N_R \times L$
E[k,m]	Channel-estimation error	$N_R \times L$
F[k,m]	Linear equalization matrix	$L \times N_R$
$\hat{s}$[k,m]	Equalized layer estimate	$L \times 1$

Table 3. Main variables of the end-to-end model.

5.2 PDSCH Transmitter Mathematical Model

The transmitter model starts from the transport block and produces one time-domain OFDM waveform per transmit antenna. The input bits are protected by CRC attachment and LDPC coding, rate matched to the allocated PDSCH resources, and scrambled. Scrambling adds a pseudo-random Gold sequence p[i], defined per TS 38.211 [2], to the coded bit sequence c[i] modulo two:

$$\tilde{c}[i] = (c[i] + p[i]) \bmod 2 \tag{28}$$

Scrambling randomizes the transmitted bits, which keeps the transmitted spectrum well behaved and separates users and channels statistically. The scrambled bits are then mapped to QAM symbols. If the modulation order is M, each symbol carries Qm = log2(M) bits, with QPSK, 16-QAM, 64-QAM, and 256-QAM corresponding to Qm = 2, 4, 6, and 8, as in equation (12). Group by group, the mapper takes Qm scrambled bits and produces one constellation symbol:

$$d[q] = map_{QAM}(\tilde{c}[qQ_m], \tilde{c}[qQ_m+1], ..., \tilde{c}[qQ_m+Q_m-1]), \quad d[q] \in S_M \tag{29}$$

where SM is the normalized constellation set of equation (14). The QAM symbols are arranged into the spatial layers. For L layers, the layer vector on subcarrier k and OFDM symbol m is

$$s[k,m] = [s_1[k,m], s_2[k,m], ..., s_L[k,m]]^T \tag{30}$$

The layer vector is mapped to the transmit antennas through the digital precoder of equation (26). Two precoding schemes are implemented. In SVD-based eigenbeamforming, the channel matrix is decomposed as

$$H[k,m] = U[k,m]\, \Sigma[k,m]\, V^H[k,m] \tag{31}$$

and the precoder W[k,m] is built from the L dominant right singular vectors, the first L columns of V[k,m]. This sends each layer along one of the strongest spatial directions of the channel. For maximum-ratio transmission with a single layer and a single receive antenna, the channel to that antenna is the 1 by NT row vector h[k,m], and the beamforming vector is

$$w[k,m] = h^H[k,m] / ||h[k,m]|| \tag{32}$$

which aligns the transmitted phases so the antenna contributions add constructively at the receiver. The precoded frequency-domain symbols are inserted into the OFDM resource grid together with the DM-RS symbols. For transmit antenna p, the time-domain OFDM symbol is generated by the unitary N-point IFFT of equation (6):

$$x_{p,m}[n] = (1/\sqrt{N}) \sum_{k=0}^{N-1} X_p[k,m]\, e^{j2\pi kn/N} \tag{33}$$

A cyclic prefix is added before transmission, exactly as in equation (10). When the channel delay spread stays within the cyclic-prefix duration, the linear convolution of the multipath channel becomes a circular convolution, and the frequency-domain model of equation (27) applies subcarrier by subcarrier, as established by equation (11). The precoded layer model itself was already shown in Figure 5, so it is not repeated here.

5.3 Channel and Receiver Observation Model

After OFDM modulation, each antenna waveform passes through the selected wireless channel: AWGN, Rayleigh fading, Rician fading, or a 3GPP TDL profile [5]. In the frequency

domain, the received vector on each resource element takes the compact form of equation (23), now with both indices:

$$y[k,m] = G[k,m]\, s[k,m] + n[k,m], \quad G[k,m] = H[k,m]\, W[k,m] \tag{34}$$

The noise vector is complex Gaussian receiver noise with independent components, following equation (20):

$$n[k,m] \sim CN(0, \sigma_n^2 I) \tag{35}$$

In the AWGN baseline, the effective channel is unity or the identity matrix, depending on the antenna configuration. In fading channels, G[k,m] changes across subcarriers, OFDM symbols, antenna ports, and spatial layers, and this is exactly why channel estimation is needed before equalization.

For PDSCH with precoded DM-RS, the receiver estimates the effective channel of the transmitted layers. The estimate is written as the true channel plus an error term:

$$\hat{G}[k,m] = G[k,m] + E[k,m] \tag{36}$$

where E[k,m] collects the channel-estimation error caused by noise, by interpolation between pilot positions, and by the limited pilot density. In ideal-channel reference simulations, the true G[k,m] is passed directly to the equalizer; comparing the two modes separates the receiver-algorithm loss from the channel-estimation loss, and this separation is used in the results of Chapter 11.

5.4 Equalization and Detection Model

The equalizer estimates the transmitted layer vector from the received vector and the estimated effective channel. On each resource element, the equalized layer estimate follows equation (24):

$$\hat{s}[k,m] = F[k,m]\, y[k,m] \tag{37}$$

For zero-forcing equalization, the matrix F[k,m] inverts the estimated effective channel:

$$F_{ZF}[k,m] = (\hat{G}^H[k,m]\, \hat{G}[k,m])^{-1}\, \hat{G}^H[k,m] \tag{38}$$

ZF removes the spatial interference completely when the effective channel is well conditioned, and it requires NR ≥ L. Its weakness is noise: when the channel has small singular values, the inverse amplifies the noise strongly. The MMSE equalizer includes the noise variance in the design and avoids this problem:

$$F_{MMSE}[k,m] = (\hat{G}^H[k,m]\, \hat{G}[k,m] + \sigma_n^2 I)^{-1}\, \hat{G}^H[k,m] \tag{39}$$

MMSE balances interference suppression against noise enhancement, and it performs better than ZF at low and medium SNR; at high SNR the two converge [10]. With unit-power

symbols, equation (14), this form needs no extra scaling. The full derivations of both equalizers are given in Chapter 7.

After equalization, each layer symbol z is demapped into soft bit values. The max-log LLR approximation for bit position i is

$$LLR_i(z) \approx (1/\sigma_{eff}^2) \left[\min_{a \in S_i^0} |z - a|^2 - \min_{a \in S_i^1} |z - a|^2 \right] \quad (40)$$

where Si0 and Si1 are the constellation subsets whose bit i equals 0 and 1, and σeff2 is the effective post-equalization noise variance of the layer. The soft bits go to the LDPC decoder, and the decoded transport block is accepted only if the CRC check passes. This makes BLER the main coded-link reliability metric, while BER, EVM, and post-equalization SINR describe the symbol-level and receiver-level behavior.

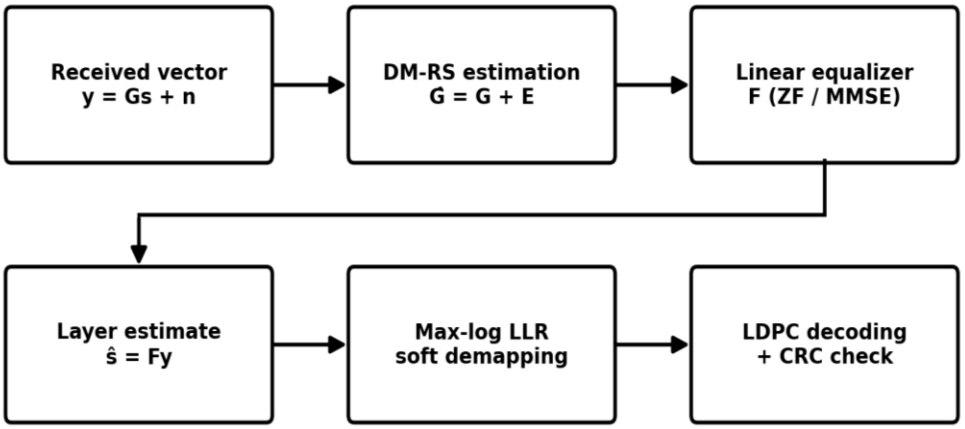


Figure 12. Receiver observation, equalization, and detection model.

The receiver side of the model appears in Figure 12. The received vector y[k,m] enters the chain, the DM-RS-based estimator produces the effective channel estimate with its error term E, and the linear equalizer, ZF or MMSE, forms the layer estimate. The max-log soft demapper converts the equalized symbols into LLR values, and the LDPC decoder with the final CRC check completes the detection. The error term E is the quantity that separates ideal-channel operation from estimated-channel operation in the simulations.

5.5 Link-Level Performance Model

The simulator evaluates performance over multiple SNR points and Monte Carlo trials, following the flow of Figure 8. For every SNR value, new transport blocks, scrambling states, channel realizations, and noise samples are generated. The same transmitter and receiver chain is used for every channel model, so the effects of AWGN, Rayleigh, Rician, and 3GPP TDL propagation can be compared fairly. The bit error rate counts the errored information bits:

$$BER = N_{bit,error} / N_{bit,total} \quad (41)$$

The block error rate follows directly from the CRC result:

$$BLER = N_{block,error} / N_{block,total} \tag{42}$$

The RMS error vector magnitude compares the equalized symbols with the transmitted ones:

$$EVM_{RMS} = \sqrt{(\Sigma_q |\hat{s}[q] - s[q]|^2 / \Sigma_q |s[q]|^2)} \tag{43}$$

and it is usually reported as a percentage. The achieved throughput divides the successfully decoded information bits by the simulated transmission time:

$$Throughput = N_{success} / T_{sim} \tag{44}$$

For the capacity reference, the MIMO channel capacity on subcarrier k with SNR ρ is [7]

$$C[k] = \log_2 \det(I_{N_R} + (\rho/N_T) H[k] H^H[k]) \tag{45}$$

These metrics connect the mathematical model to the simulation results of Chapter 11. BER and BLER measure reliability, EVM measures modulation accuracy, throughput and spectral efficiency measure useful data delivery, post-equalization SINR measures receiver output quality, NMSE measures channel-estimation accuracy, and capacity gives the upper-bound reference for MIMO performance.

The MATLAB implementation uses this model as the primary simulation and verification chain. The Python implementation mirrors the same signal flow and the same equations using NumPy and SciPy matrix operations, which allows independent cross-checking of the transmitter processing, the channel application, the receiver equalization, and the performance-metric calculation.

Chapter 6. Wireless Channel Models

This Sections defines the wireless channel models used by the simulator: AWGN, Rayleigh fading, Rician fading, and the 3GPP TDL profiles. Each model is given with its mathematical definition, its purpose in the evaluation, and its effect on the receiver processing. The chapter closes with the common channel configuration that connects every model to the end-to-end system of Chapter 5. In the time-domain scalar equations of this chapter, the noise sample is written w[n], to keep it clearly separate from the sample index n; its statistics are exactly those of the noise model in equations (18) to (21).

6.1 AWGN Channel

The AWGN channel is the baseline propagation model, used to validate the simulator before any fading is enabled. It adds complex receiver noise and nothing else: no multipath, no Doppler spread, no delay spread, and no frequency selectivity. The discrete-time received baseband sample is

$$y[n] = x[n] + w[n] \tag{46}$$

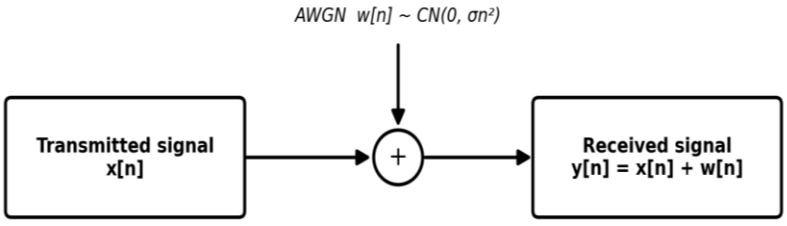
where $w[n] \sim CN(0, \sigma n^2)$ follows the noise model of equations (19) and (20), and the noise variance for a target SNR follows equation (21). After OFDM demodulation, the received subcarrier is simply

$$Y[k] = X[k] + N[k] \tag{47}$$

For a SISO AWGN reference, the channel response is unity. For a MIMO baseline verification case with an equal number of transmit and receive antennas, the channel can be set to the identity matrix:

$$H_{AWGN}[k] = 1 \quad (SISO), \qquad H_{AWGN}[k] = I \quad (MIMO,\ N_T = N_R) \tag{48}$$

Figure 13 illustrates the AWGN verification path. The transmitted OFDM waveform passes through a unity channel response, and complex noise is added according to the selected SNR. This configuration checks the modulation, the OFDM processing, the demodulation, the LDPC decoding, the CRC verification, and the metric calculation before any fading-channel simulation is executed. It is also the configuration in which the simulated BER curves are compared against closed-form theory in Chapter 10.



Channel response H = 1: noise only, no fading, no delay spread.

Figure 13. AWGN channel model used for baseline simulator verification.

The AWGN model adds one noise sample to each transmitted sample and changes nothing else. The channel response is one, so the received signal is the transmitted signal plus complex Gaussian noise with the variance set by the target SNR of equation (21). Every block of the transmitter and the receiver can be verified in this configuration, because any error that appears cannot be blamed on the channel.

6.2 Rayleigh Fading Channel

The Rayleigh fading channel models non-line-of-sight propagation, where the received signal is the sum of many scattered components and no dominant direct path exists. In this simulator, Rayleigh fading tests the robustness of the DM-RS channel estimation, the ZF and MMSE equalization, the soft demodulation, and the LDPC decoding under rich-scattering conditions. For flat Rayleigh fading, the received sample is

$$y[n] = h[n]\, x[n] + w[n] \tag{49}$$

The fading coefficient has independent Gaussian real and imaginary parts [8], [9]:

$$h[n] = h_I[n] + j\, h_Q[n], \quad h_I,\, h_Q \sim N(0,\, 1/2) \tag{50}$$

so the coefficient is a unit-power complex Gaussian:

$$h[n] \sim CN(0,\, 1), \quad E\{\, |h[n]|^2 \,\} = 1 \tag{51}$$

The unit power keeps the average received SNR equal to the configured SNR, so the fading changes the distribution of the channel but not its mean energy. For OFDM transmission over a frequency-selective Rayleigh channel, the received subcarrier is

$$Y[k,m] = H[k,m]\, X[k,m] + N[k,m] \tag{52}$$

and for MIMO-OFDM transmission the vector observation of equation (27) applies with an NR by NT matrix whose entries are independent CN(0, 1) coefficients:

$$y[k,m] = H[k,m]\, x[k,m] + n[k,m] \tag{53}$$

The matrix H[k,m] changes across subcarriers, OFDM symbols, and antenna pairs from one realization to the next. Unlike the AWGN case, the channel response is no longer unity, so the receiver must estimate it from the DM-RS symbols and equalize before demapping. The Rayleigh model is shown in Figure 14.

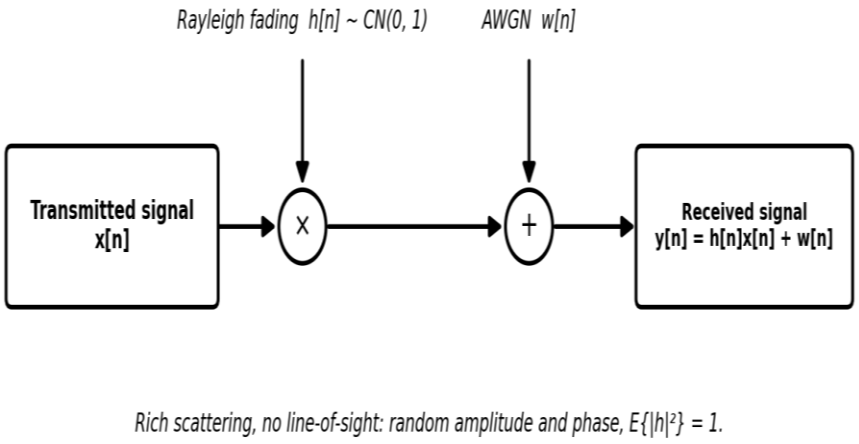


Figure 14. Rayleigh fading channel model used for NLOS rich-scattering evaluation.

The transmitted signal is multiplied by the random fading coefficient h[n] and the noise is added afterwards. The coefficient is a zero-mean, unit-power complex Gaussian, which gives the received amplitude a Rayleigh distribution and the phase a uniform distribution. Deep fades occur whenever the scattered components add destructively, and these fades are what the channel estimation and the equalization must survive.

6.3 Rician Fading Channel

The Rician fading channel applies when a dominant line-of-sight component exists together with scattered multipath. This model fits outdoor small-cell links, fixed wireless access, millimeter-wave propagation, and satellite-related channels, where the direct path can be strong but multipath still shapes the received signal. The normalized Rician coefficient combines a deterministic and a random part [8], [9]:

$$h[n] = \sqrt{(K/(K+1))}\, h_{LOS} + \sqrt{(1/(K+1))}\, h_{NLOS}[n] \tag{54}$$

where hLOS is the deterministic line-of-sight component with |hLOS| = 1, and hNLOS[n] ~ CN(0, 1) is the scattered component; with these normalizations, $E\{|h[n]|^2\} = 1$, so the average received power stays independent of K. The Rician K-factor is the power ratio of the two parts:

$$K = P_{LOS} / P_{NLOS} \tag{55}$$

and it is usually quoted in decibels:

$$K_{dB} = 10\, log_{10}(K) \tag{56}$$

When K goes to zero, the model becomes Rayleigh fading. As K grows, the deterministic part dominates and the fading becomes milder. In the MIMO-OFDM case, the same decomposition applies to the whole channel matrix:

$$H[k,m] = \sqrt{(K/(K+1))}\, H_{LOS}[k,m] + \sqrt{(1/(K+1))}\, H_{NLOS}[k,m] \tag{57}$$

The receiver chain does not change at all between the Rayleigh and the Rician cases: the DM-RS estimation, the ZF and MMSE equalization, the soft demodulation, the LDPC decoding, and the CRC check are applied unchanged. Only the statistics of the channel matrix differ, and that difference appears directly in the BER, BLER, and NMSE results. Figure 15 presents the two-branch structure of the model.

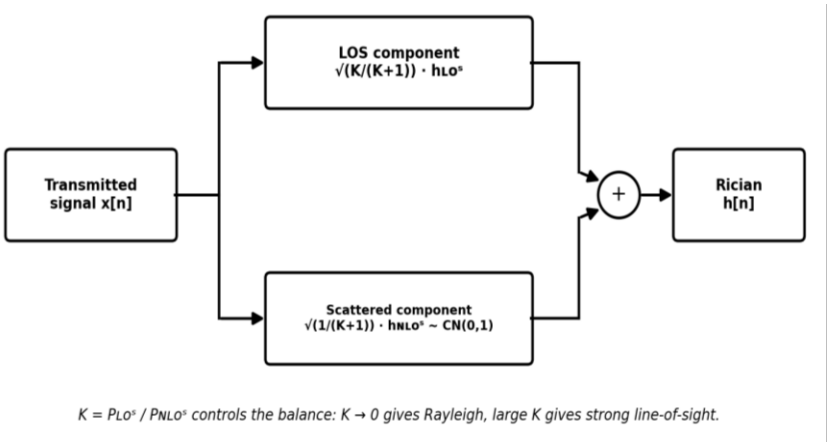


Figure 15. Rician fading channel model with LOS and scattered NLOS components.

The transmitted signal splits into two branches. The upper branch carries the deterministic line-of-sight component with weight $\sqrt{(K/(K+1))}$, and the lower branch carries the random scattered component with weight $\sqrt{(1/(K+1))}$. Their sum forms the Rician coefficient. The K-factor moves the model continuously between pure Rayleigh fading at K = 0 and an almost constant line-of-sight channel at large K, which lets the simulator sweep the full range of fading severity with one parameter.

6.4 3GPP TDL Channel Models

The 3GPP tapped-delay-line models evaluate the simulator under standardized multipath conditions [5]. A TDL profile describes the channel as a set of Ntap delayed taps, each with a defined delay, an average power, and a fading coefficient. The taps create delay spread, and the delay spread creates frequency selectivity across the OFDM subcarriers, which is exactly the condition the PDSCH receiver must handle. The baseband channel impulse response is

$$h(t, \tau) = \Sigma_{l=0}^{Ntap-1} \alpha_l(t)\, \delta(\tau - \tau_l) \quad (58)$$

where $\alpha l(t)$ is the complex coefficient of tap l and τl is its delay. Each tap has average power $E\{|\alpha l|^2\} = pl$, and the tap powers are normalized so that they sum to one, which keeps the average channel power at one, exactly as in the Rayleigh and Rician cases. With subcarrier spacing Δf, the frequency response on subcarrier k and OFDM symbol m follows by Fourier transform of the taps:

$$H[k,m] = \Sigma_{l=0}^{Ntap-1} \alpha_l[m]\, e^{-j2\pi k \Delta f \tau_l} \quad (59)$$

and the MIMO-OFDM observation keeps the standard form of equation (27), with one NR by NT matrix per resource element:

$$y[k,m] = H[k,m]\, x[k,m] + n[k,m] \tag{60}$$

Under the quasi-static assumption of Section 2.4, the tap coefficients αl[m] stay constant within one slot; when Doppler is enabled, they vary from symbol to symbol according to the configured Doppler spectrum. The TDL profiles differ in their delay-spread character: TDL-A, TDL-B, and TDL-C are non-line-of-sight profiles with different delay spreads, while TDL-D and TDL-E contain a dominant line-of-sight tap and behave like Rician channels. In this project, TDL-A, TDL-C, and TDL-D are the baseline standardized profiles, and TDL-B and TDL-E are extension cases. The tap delays of TR 38.901 are normalized and are scaled by the configured RMS delay spread, so one profile covers many deployment scenarios. The tapped-delay-line structure appears in Figure 16.

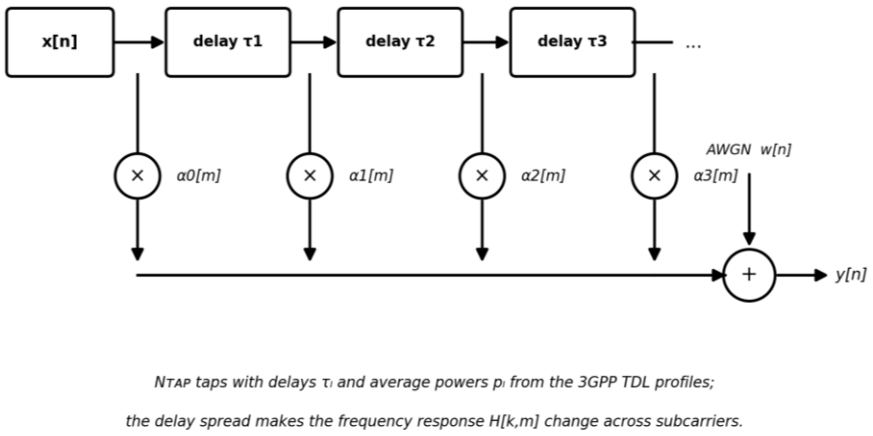


Figure 16. 3GPP TDL channel model used for frequency-selective MIMO-OFDM simulation.

The transmitted signal passes along a delay line. At each tap position, the delayed signal is weighted by the complex tap coefficient αl[m], and the weighted taps are summed before the receiver noise is added. The tap delays and average powers come from the selected TR 38.901 profile. Because the taps arrive at different delays, the frequency response of equation (59) changes across the subcarriers, and the OFDM receiver observes a different complex gain on each allocated resource element.

6.5 Channel Configuration Used in the Simulator

The simulator uses one receiver architecture for every supported channel model, so the effect of the propagation environment can be measured independently of the receiver implementation. The transmitter generates the PDSCH-based MIMO-OFDM waveform, and the selected channel model decides how that waveform is distorted before the receiver chain runs. The channel selector is

$$H_{ch}[k,m] \in \{ I, H_{Rayleigh}[k,m], H_{Rician}[k,m], H_{TDL}[k,m] \} \quad (61)$$

With precoding enabled, the received vector and the effective layer channel follow directly from equations (27) and (34):

$$y[k,m] = H_{ch}[k,m]\, W[k,m]\, s[k,m] + n[k,m], \quad G[k,m] = H_{ch}[k,m]\, W[k,m] \quad (62)$$

For precoded PDSCH DM-RS, the receiver estimates the effective layer channel directly, as in equation (36), and the linear equalizer of equations (38) and (39) produces the layer estimate:

$$\hat{s}[k,m] = F[k,m]\, y[k,m], \quad F \in \{ F_{ZF}, F_{MMSE} \} \quad (63)$$

Path loss and shadow fading are not modeled as separate blocks in the baseline simulations. Their effect is carried by the configured received SNR, following the link-level convention stated in Section 3.4. This keeps the chapter focused on PHY algorithm verification rather than large-scale coverage prediction. For every channel model, the simulator computes the same performance metrics: BER, BLER, EVM, throughput, spectral efficiency, post-equalization SINR, channel-estimation NMSE, and MIMO capacity. The overall channel processing structure, with the selectable model and the added receiver noise, was already shown in Figure 6, and it is not repeated here. Table 4 summarizes the four models, their purposes, and what each one demands from the receiver.

Channel model	**Purpose in simulator**	**Main effect**	**Receiver requirement**
AWGN	Baseline verification	Noise only	Standard demodulation and decoding
Rayleigh	NLOS fading evaluation	Random amplitude and phase fading	DM-RS channel estimation and equalization
Rician	LOS plus scattering evaluation	K-factor controlled fading severity	DM-RS channel estimation and equalization
3GPP TDL	Standardized multipath evaluation	Delay spread and frequency selectivity	DM-RS channel estimation and per-subcarrier equalization

Table 4. Channel models used in the proposed simulator.

Chapter 7. Physical Layer Signal Processing Algorithms

This Section presents the signal-processing algorithms of the simulator and derives the estimators and equalizers that the earlier chapters introduced by name. The system model is the one built in Chapters 4 to 6: on every resource element, the received vector follows equations (27) and (34), the effective layer channel is G[k,m] = H[k,m]W[k,m], and the receiver works on the estimated effective channel of equation (36). The same algorithmic chain runs unchanged over the AWGN, Rayleigh, Rician, and 3GPP TDL channels of Chapter 6, so the channel effect is measured without touching the transmitter or receiver structure. Each algorithm is given as a derivation, as a pseudocode block, and, where a new structure appears, as a figure.

7.1 End-to-End PHY Algorithm

The end-to-end algorithm converts a transport block into a precoded multi-antenna OFDM waveform and recovers the transmitted bits after the channel distortion. Its mathematical path was fully defined in Chapter 5 and shown in Figure 11, so no equation is repeated here; Algorithm 1 states the complete procedure step by step, in the order in which the implementation executes it. Every step carries the number of the equation it implements, which gives the direct traceability from the code back to the mathematics.

Algorithm 1: End-to-end PDSCH link-level simulation (one Monte Carlo trial)
1: Generate the transport block b, equation (25).
2: Attach the CRC, apply LDPC encoding and rate matching [3], and scramble, equation (28).
3: Map the scrambled bits to QAM symbols, equation (29), and arrange them into L layers, equation (30).
4: Precode the layer vector to the antennas, x = Ws, equation (26), and map data and DM-RS onto the resource grid, equation (64).
5: Apply the unitary IFFT, equation (33), and insert the cyclic prefix, equation (10).
6: Apply the selected channel model of Chapter 6 and add noise, equations (35) and (61).
7: Remove the cyclic prefix, apply the FFT, and demap the resource grid.
8: Estimate the effective channel from the DM-RS, Algorithm 3, equations (65) to (69).
9: Equalize with ZF or MMSE, Algorithm 4, equations (37) to (39).
10: Compute the LLR values, equation (40), decode the LDPC code, and check the CRC, equation (77).
11: Accumulate the metrics: BER, BLER, EVM, throughput, SINR, and NMSE, equations (41) to (44), (76), (78).

7.2 PDSCH Transmitter Algorithm

The transmitter algorithm follows the chain of Figure 3 and the equations of Section 5.2: the transport block of equation (25) passes through CRC attachment, LDPC encoding, rate matching, scrambling per equation (28), QAM mapping per equation (29), layer mapping per equation (30), and digital precoding per equation (26), with the layer count limited by L ≤

min(NT, NR). None of these operations needs a new equation; the one step not yet written formally is the resource-grid mapping. For antenna port p, the grid value on resource element (k, m) is

$$X_p[k,m] = d(k,m) \ \text{if } (k,m) \in \text{data REs}, \quad r(k,m) \ \text{if } (k,m) \in \text{DM-RS REs}, \quad 0 \ \text{otherwise} \qquad (64)$$

where d(k,m) is the precoded data symbol and r(k,m) is the precoded DM-RS symbol of the port. The grid then enters the unitary IFFT of equation (33), and the cyclic prefix of equation (10) completes the waveform. Algorithm 2 lists the full transmitter procedure.

Algorithm 2: PDSCH transmitter processing
1: Input: transport block b, MCS, antenna and layer configuration.
2: Attach the transport-block CRC; segment into code blocks with code-block CRCs if needed [3].
3: LDPC-encode each code block, rate match, and concatenate.
4: Scramble the coded bits with the Gold sequence, equation (28) [2].
5: Map groups of Qm bits to QAM symbols, equations (12) and (29).
6: Arrange the symbols into L layers, equation (30).
7: Compute the precoder W: SVD eigenbeamforming, equation (31), or MRT, equation (32).
8: Precode, x = Ws, equation (26), and fill the resource grid, equation (64).
9: Apply the unitary IFFT per antenna, equation (33), and insert the cyclic prefix, equation (10).
10: Output: multi-antenna time-domain OFDM waveform.

7.3 DM-RS-Based Channel Estimation

The receiver estimates the effective channel from the received DM-RS resource elements. Because the DM-RS is precoded together with the data, the estimate targets G[k,m] directly, never the full antenna channel H[k,m]. For a single layer, the received DM-RS sample is Y = G r + N with the known reference symbol r, so the least-squares estimate at a pilot position is the direct division

$$\hat{G}_{LS}[k,m] = Y_{DMRS}[k,m] \,/\, r[k,m] \qquad (65)$$

Substituting Y = G r + N shows exactly what the LS estimate contains:

$$\hat{G}_{LS}[k,m] = G[k,m] + N[k,m]/r[k,m] \qquad (66)$$

so the LS error is pure scaled noise with variance $\sigma n^2/|r|^2$, which equals σn^2 for unit-power DM-RS. LS is unbiased and simple, but it passes the noise straight through. For multiple layers, the pilot observation collects the layer references in the matrix SDMRS, and the LS solution generalizes to

$$\hat{G}_{LS}[k,m] = Y_{DMRS}[k,m]\, S_{DMRS}^{H} \left(S_{DMRS}\, S_{DMRS}^{H} \right)^{-1} \qquad (67)$$

With the orthogonal Type-1 DM-RS design, the product SDMRS SDMRSH is diagonal, and the matrix solution reduces to independent per-layer divisions. The MMSE estimator improves

on LS by using the frequency correlation of the channel. Treating the LS estimates along frequency as a vector, the linear MMSE (Wiener) estimate is [10]

$$\hat{G}_{MMSE} = R_G \left(R_G + \sigma_n^2 I \right)^{-1} \hat{G}_{LS} \tag{68}$$

where RG is the channel correlation matrix across the pilot subcarriers. The MMSE estimator acts as a smoother: where the channel is strongly correlated it averages the noisy LS values, and its mean square error is never worse than LS. In the flat-fading limit, equation (68) collapses to a simple scalar gain smaller than one that shrinks the noisy estimate toward zero. The LS and MMSE estimates exist only at the DM-RS positions; interpolation over frequency and, when needed, over OFDM symbols extends them to every data resource element:

$$\hat{G}[k,m] = Interp\{ \hat{G}[k_{DMRS}, m_{DMRS}] \} \tag{69}$$

A perfect-channel reference mode passes the true G[k,m] to the equalizer, which separates the channel-estimation loss from the equalizer and decoder behavior, as introduced with equation (36). The complete pipeline is summarized in Algorithm 3, and its structure appears in Figure 17.

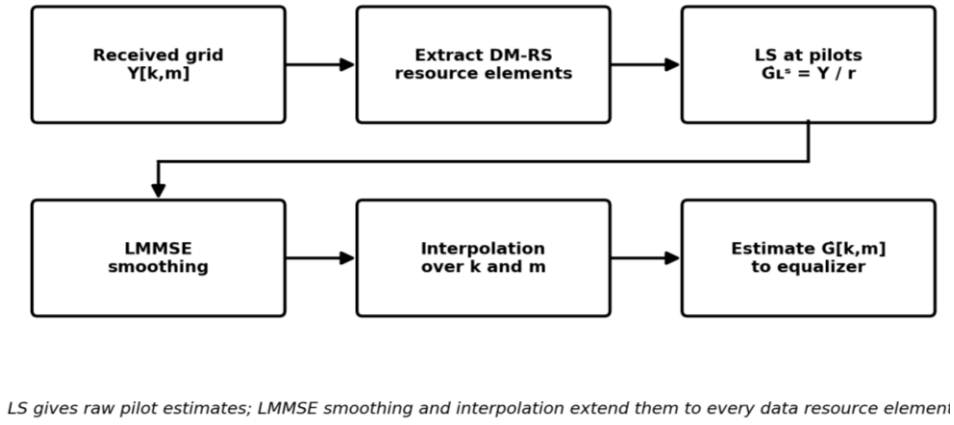


Figure 17. DM-RS-based effective channel estimation pipeline.

Figure 17 presents the estimation pipeline from the received grid to the equalizer input. The DM-RS resource elements are extracted, the LS estimate of equation (65) is formed at every pilot, the LMMSE smoothing of equation (68) suppresses the noise on the pilot estimates, and the interpolation of equation (69) extends the result to all data resource elements. The output is the estimated effective channel used by the equalizer.

Algorithm 3: DM-RS-based channel estimation
1: Input: received resource grid Y[k,m], DM-RS positions and reference symbols r.
2: Extract the received DM-RS resource elements.
3: Compute the LS estimates at the pilots: single layer by equation (65), multiple layers by equation (67).
4: Apply LMMSE smoothing across the pilot subcarriers, equation (68).
5: Interpolate over frequency, and over OFDM symbols when the channel varies within the slot, equation (69).
6: Output: estimated effective channel $\hat{G}[k,m]$ for every allocated resource element.

7.4 Linear MIMO Equalization: Derivations

The equalizers stated in equations (38) and (39) are now derived. The zero-forcing equalizer answers a least-squares question: given the observation $y = \hat{G}s + n$, which layer vector explains the observation best? The cost is the squared residual

$$J_{ZF}(s) = || y - \hat{G}\, s ||^2 \tag{70}$$

Setting the gradient with respect to s to zero gives the normal equations

$$\hat{G}^H \hat{G}\, \hat{s} = \hat{G}^H y \tag{71}$$

whose solution is exactly the ZF equalizer of equation (38); it exists when $NR \geq L$ and the columns of $\hat{G}$ are independent. ZF treats the noise as if it did not exist, which is why it amplifies noise whenever the channel matrix is poorly conditioned. The MMSE equalizer fixes this by minimizing the error itself, not the residual:

$$J_{MMSE}(F) = E\{ || s - F y ||^2 \} \tag{72}$$

With unit-power layers, equation (14), and noise of equation (35), the orthogonality principle gives the Wiener solution

$$F = \hat{G}^H (\hat{G}\, \hat{G}^H + \sigma_n^2\, I)^{-1} \tag{73}$$

and the standard matrix-inversion identity turns equation (73) into the equivalent, computationally smaller form of equation (39), which inverts an L by L matrix instead of an NR by NR one [10]. At high SNR the $\sigma n^2 I$ term vanishes and MMSE converges to ZF; at low SNR the term dominates and MMSE behaves like a matched filter. Figure 18 shows the equalization stage with its two inputs.

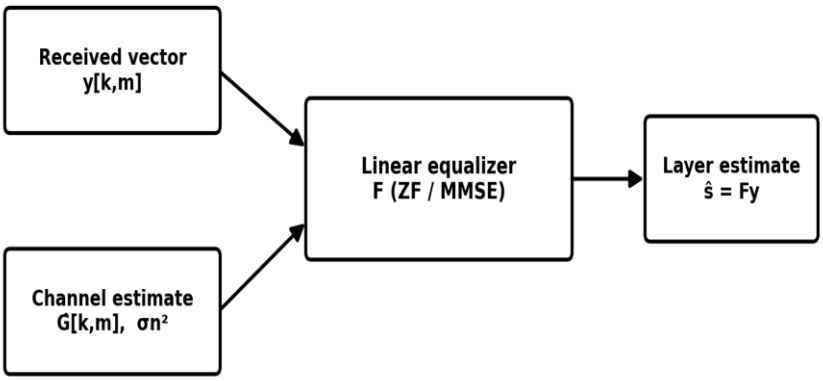


Figure 18. Linear MIMO equalization stage with ZF and MMSE options.

The equalization stage is shown in Figure 18. Two inputs feed the equalizer: the received vector y[k,m] and the channel estimate $\hat{G}$[k,m] together with the noise variance. The equalizer matrix is built either as ZF, equation (38), or as MMSE, equation (39), and the output is the layer estimate of equation (37). The note under the diagram records the trade-off that the derivations above make precise: ZF removes interference completely but amplifies noise, while MMSE balances the two.

7.5 Soft Demodulation, Detection Quality, and Metrics

After equalization, each layer symbol is demapped into LLR values using the max-log rule of equation (40). The rule needs the effective post-equalization noise variance of the layer, and this variance now has an exact expression. The layer error decomposes into a residual-interference part and a filtered-noise part:

$$e = \hat{s} - s = (F \hat{G} - I) s + F n \tag{74}$$

so the effective noise variance of layer l is the power of its error row:

$$\sigma_{eff,l}^2 = || (F \hat{G} - I)_l ||^2 + \sigma_n^2 || F_l ||^2 \tag{75}$$

where the subscript l selects the l-th row. The same decomposition gives the post-equalization SINR of the layer, the metric listed since Chapter 2:

$$SINR_l = | [F \hat{G}]_{ll} |^2 / (\Sigma_{j \neq l} | [F \hat{G}]_{lj} |^2 + \sigma_n^2 || F_l ||^2) \tag{76}$$

The LLR values pass through rate recovery to the LDPC decoder [3], and the transport block is accepted when the CRC check clears:

$$CRC(\hat{b}) = 0 \tag{77}$$

The link metrics BER, BLER, EVM, and throughput follow equations (41) to (44) unchanged. One promised metric still lacked a formal definition: the channel-estimation NMSE, which compares the estimate against the true channel over the allocated resource elements:

$$NMSE = E\{ || \hat{G} - G ||^2 \} / E\{ || G ||^2 \} \quad (78)$$

The NMSE measures the estimation pipeline of Section 7.3 on its own, before the equalizer and the decoder, and it is the metric that separates the LS and MMSE estimators in the results of Chapter 11. Algorithm 4 collects the complete receiver procedure.

Algorithm 4: Receiver equalization, detection, and metric extraction
1: Input: received grid Y[k,m], estimated channel Ĝ[k,m] from Algorithm 3, noise variance σn^2.
2: Build the equalizer per resource element: ZF by equations (70), (71), (38), or MMSE by equations (72), (73), (39).
3: Form the layer estimates ŝ = Fy, equation (37).
4: Compute the per-layer effective noise variance, equation (75), and the post-equalization SINR, equation (76).
5: Demap each layer symbol into LLR values with the max-log rule, equation (40).
6: Descramble, recover the rate matching, and run the LDPC decoder [3].
7: Accept the transport block if the CRC clears, equation (77).
8: Update BER, BLER, EVM, throughput, SINR, and NMSE, equations (41) to (44), (76), (78).

7.6 Algorithm-to-Implementation Mapping

The MATLAB and Python implementations use the same algorithmic interfaces, so the two simulators can be compared under identical assumptions. MATLAB is the primary verification platform, and Python is the independent reference implementation. Table 5 maps each algorithm block of this chapter to its main inputs, its main outputs, and the metric that verifies it; the implementation chapters that follow translate these blocks into MATLAB and Python modules while keeping the same signal definitions, the same channel-estimation interface, the same equalizer equations, and the same metric calculations.

Algorithm block	**Main input**	**Main output**	**Verification metric**
PDSCH transmitter (Algorithm 2)	Transport block bits, MCS, antenna and layer configuration	Precoded OFDM waveform	Reference waveform consistency
DM-RS channel estimation (Algorithm 3)	Received grid and known DM-RS symbols	Estimated effective channel	Channel-estimation NMSE, equation (78)
ZF/MMSE equalization (Algorithm 4)	Received vector and estimated channel	Equalized layer symbols	EVM and post-equalization SINR, equation (76)
Soft demodulation and LDPC decoding	Equalized symbols and noise estimate	Decoded transport block	BER and BLER
Metric extraction	Decoded bits, reference bits, symbol estimates	BER, BLER, EVM, throughput, SINR, NMSE	Monte Carlo result consistency

Table 5. Algorithm-to-implementation mapping.

Chapter 8. MATLAB Implementation

This Section describes the MATLAB implementation of the simulator. The implementation translates the mathematical model of Chapter 5, the channel models of Chapter 6, and the algorithms of Chapter 7 into a modular link-level simulator built around one simulation controller, one configuration file, and independent transmitter, channel, receiver, and metric modules. MATLAB is the primary simulation and verification platform of the project; the executed result campaigns of Chapter 11 and the coded-mode campaign both pass through the same acceptance gate of Chapter 10. The code itself lives in the project repository; this chapter presents the structure, the interfaces, and the few implementation-specific relations that the earlier chapters did not need.

8.1 Implementation Architecture and Numerology

The module structure follows the algorithm blocks of Table 5 directly: one function group per block, with clean interfaces between them. This keeps the simulator readable, testable at every stage, and open for extension to more antenna configurations, modulation orders, channel models, and receiver algorithms. Two implementation relations fix the numerology in code. The number of active subcarriers follows from the number of resource blocks:

$$N_{SC} = 12\, N_{RB} \tag{79}$$

For the baseline FR1 configuration of Table 2, with 30 kHz subcarrier spacing and 273 resource blocks [6], this gives 3276 active subcarriers, and the 4096-point FFT of the baseline satisfies N ≥ NSC with a power-of-two size for fast transforms. The discrete-time sample rate then follows from the FFT size and the subcarrier spacing:

$$F_s = N\, \Delta f \tag{80}$$

With N = 4096 and Δf = 30 kHz, the baseband sampling rate is 122.88 MHz. The same parameter set is read by the transmitter, the channel, and the receiver functions, so the numerology can never drift between modules. Table 6 lists the modules and their responsibilities, and Figure 19 shows how they connect.

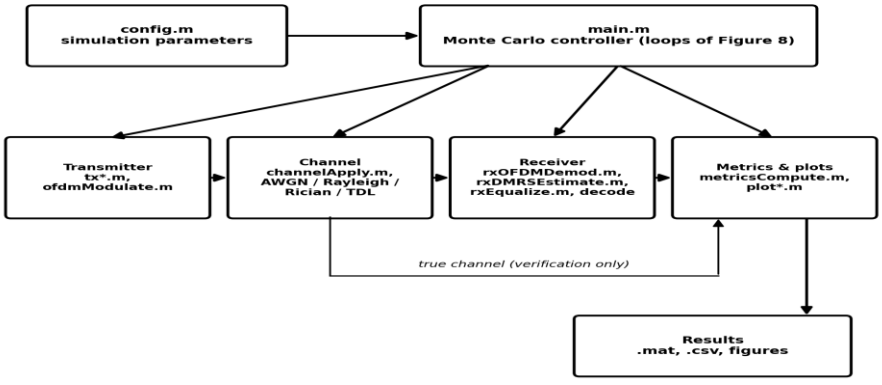


Figure 19. MATLAB implementation architecture.

The architecture of the MATLAB simulator is shown in Figure 19. The configuration file feeds the Monte Carlo controller, which owns the SNR and trial loops of Figure 8 and calls the four module groups in signal order: transmitter, channel, receiver, and metrics. The thin path from the channel to the metrics module carries the true channel response, which is used only for verification, for the NMSE of equation (78) and for the ideal-channel reference mode of equation (36); the receiver itself never sees it. The metrics module writes the results to MAT and CSV files and generates the figures.

Module	Main files	Role in the simulator
Simulation control	main.m, config.m	Defines the NR numerology, the antenna configuration, the SNR range, the channel model, the random seed, and runs the Monte Carlo loops.
Transmitter	txGenerateTB.m, txCRCEncode.m, txLDPCEncode.m, txRateMatch.m, txScramble.m, txQAMMap.m, txLayerMap.m, txPrecode.m, ofdmModulate.m	Generates the coded and precoded PDSCH MIMO-OFDM waveform, Algorithm 2.
Channel	channelApply.m, channelAWGN.m, channelRayleigh.m, channelRician.m, channelTDL.m	Applies the selected channel of Chapter 6 and the receiver noise, equations (35) and (61).
Receiver	rxOFDMDemod.m, rxDMRSEstimate.m, rxEqualize.m, rxLLRDemap.m, rxLDPCDecode.m, rxCRCCheck.m	Runs OFDM demodulation, channel estimation (Algorithm 3), equalization and detection (Algorithm 4).
Metrics and plotting	metricsCompute.m, metricsAggregate.m, plotBER.m, plotBLER.m, plotEVM.m, plotThroughput.m	Computes and stores BER, BLER, EVM, SINR, NMSE, throughput, spectral efficiency, and capacity.

Table 6. MATLAB implementation modules and responsibilities.

8.2 Configuration and Simulation Control

The file config.m defines every simulation parameter before any waveform processing starts: the carrier numerology, the bandwidth, the modulation order, the number of layers, the transmit and receive antenna counts, the channel type, the SNR vector, the number of Monte Carlo trials, and the random seed. The main script reads this configuration once and executes the nested loops of Figure 8 over the SNR points and the independent realizations. For each SNR point, the receiver noise variance generalizes equation (21) to a measured received power:

$$\sigma_n^2 = P_{rx} / 10^{SNR_{dB}/10} \quad (81)$$

With the power normalizations of Chapters 4 and 5, Prx equals one and equation (81) reduces to equation (21); measuring Prx keeps the SNR calibration correct even when a configuration changes the waveform scaling. The Monte Carlo average of any metric z over the configured number of independent trials is

$$\bar{z}(SNR) = (1/N_{trial}) \Sigma_{i=1}^{Ntrial} z_i(SNR) \quad (82)$$

A simplified version of the execution loop is shown below. The final implementation keeps every processing block in its own function, so debugging can happen at the waveform level, the resource-grid level, the channel-estimation level, the equalization level, or the decoding level, one stage at a time.

```
cfg = config();
rng(cfg.seed);
for isnr = 1:length(cfg.snrDb)
    for itrial = 1:cfg.numTrials
        tx  = txPdschWaveform(cfg);
        ch  = channelApply(tx, cfg, cfg.snrDb(isnr));
        rx  = rxPdschProcess(ch, cfg);
        results = metricsUpdate(results, tx, rx, isnr);
    end
end
```

The rng seed call is the single point of randomness control required by requirement R8 of Table 1: with the same seed, the complete experiment reproduces exactly, and with different seeds the trials are statistically independent.

8.3 Transmitter Implementation

The transmitter module produces the time-domain multi-antenna waveform for the channel block, implementing Algorithm 2 step by step: random transport-block generation per equation (25), CRC attachment, LDPC coding and rate matching, scrambling per equation (28), QAM mapping per equation (29), layer mapping per equation (30), precoding per equation (26), resource-grid filling per equation (64), and OFDM modulation with the unitary IFFT of equation (33) and the cyclic prefix of equation (10). The unitary normalization is applied explicitly, because the built-in MATLAB ifft uses the non-unitary 1/N convention; without the explicit scaling, the code would silently disagree with the mathematics of Chapter 4.

The coding stage runs through an interface with two modes. In the Toolbox mode, the CRC, LDPC, and rate-matching steps use MATLAB 5G Toolbox functions [12], which implement the TS 38.212 processing [3] with base-graph selection and code-block segmentation. In the simulator-level verification mode, a simplified coding path is used for algorithm isolation, and every result produced in that mode is labeled as such. This two-mode interface is how the report keeps the compliance claim of Chapter 1 honest: TS 38.212 compliance is claimed exactly where the Toolbox functions run, and nowhere else. Waveform normalization is applied before transmission, so the SNR scaling of equation (81) stays consistent across modulation orders and antenna configurations.

8.4 Channel and Receiver Implementation

The channel interface accepts the transmitted antenna waveform and returns three things: the received waveform, the applied noise variance, and the true channel response. The true response leaves the channel block only on the verification path of Figure 19; during normal operation the receiver estimates the effective channel from the DM-RS, exactly as Chapter 7 defines. The channel implementations cover the four models of Chapter 6 and apply equation (61) per resource element with the noise of equation (35).

The receiver module implements Algorithms 3 and 4 in order: OFDM demodulation with the unitary FFT, resource-element demapping, LS estimation at the pilots per equations (65) and (67), LMMSE smoothing per equation (68), interpolation per equation (69), ZF or MMSE equalization per equations (38) and (39), the layer estimate of equation (37), the effective noise variance of equation (75), the max-log LLR of equation (40), descrambling, rate recovery, LDPC decoding, and the CRC check of equation (77). The receiver output structure stores the decoded bits, the CRC status, the equalized symbols, the estimated channel, and the per-realization metrics, so every later analysis can be reproduced from stored data.

8.5 Metric Computation and Result Storage

The metric module compares the transmitted and the recovered information and produces the numbers used in Chapter 11. All metrics follow the definitions already fixed: BER and BLER per equations (41) and (42), EVM per equation (43), throughput per equation (44), post-equalization SINR per equation (76), and channel-estimation NMSE per equation (78). Each metric is computed per Monte Carlo realization and averaged with equation (82) for every SNR point. The result structure is saved in MATLAB MAT format and exported to CSV whenever the values are needed for external plotting or for the Python cross-check of Chapter 9.

Chapter 9. Python Implementation

The Python implementation is the independent reference model, built on NumPy [13], SciPy, and Matplotlib. Its purpose is not to replace the MATLAB implementation but to verify the mathematical model, the array handling, the normalizations, the receiver algorithms, and the metric calculations in a separate numerical environment, on the same chain and scope limits of Chapters 5 to 7. Two implementations written in different environments rarely agree by accident, so agreement between them is strong evidence that both are correct; this chapter defines how that agreement is established.

9.1 Configuration and Software Architecture

Every simulation run is defined by one shared configuration tuple that both implementations read:

$$\theta_{sim} = \{ B, \Delta f, N_{RB}, N, N_T, N_R, L, M, R, SNR_{dB}, \text{channel type, seed} \} \quad (83)$$

The tuple collects the bandwidth, the subcarrier spacing, the resource-block count, the FFT size, the antenna and layer counts, the modulation order, the code rate, the SNR vector, the channel selection, and the random seed. Sharing one configuration makes both simulators deterministic and directly comparable. The Python package mirrors the MATLAB module structure of Figure 19 one for one: a main script owns the SNR and Monte Carlo loops of Figure 8, and the transmitter, channel, receiver, and metric packages own the mathematics. Table 7 lists the modules and the verification purpose of each. A simplified version of the control loop shows the parity with the MATLAB loop of Section 8.2:

```
cfg = load_config()
rng = np.random.default_rng(cfg.seed)
for snr_db in cfg.snr_db:
    for trial in range(cfg.num_trials):
        tx  = tx_pdsch_waveform(cfg, rng)
        ch  = channel_apply(tx, cfg, snr_db, rng)
        rx  = rx_pdsch_process(ch, cfg)
        results.update(tx, rx, snr_db)
```

The seeded default_rng generator is the Python counterpart of the MATLAB rng call and satisfies the same reproducibility requirement R8 of Table 1: identical seeds reproduce the experiment exactly, and different seeds give statistically independent trials.

Module	Main responsibility	Verification purpose
config.py	Stores the numerology, antenna, modulation, coding, SNR, and channel settings of equation (83).	Ensures MATLAB and Python use identical assumptions.
transmitter/	Generates the coded, scrambled, modulated, layer-mapped, precoded, and OFDM-modulated PDSCH waveform (Algorithm 2).	Checks symbol normalization and grid construction.
channel/	Applies the AWGN, Rayleigh, Rician, or TDL response of Chapter 6 and the receiver noise.	Verifies SNR calibration and channel-matrix consistency.
receiver/	Runs OFDM demodulation, DM-RS estimation (Algorithm 3), equalization and detection (Algorithm 4).	Checks receiver-algorithm correctness.
metrics/	Computes BER, BLER, EVM, SINR, NMSE, throughput, spectral efficiency, and capacity.	Confirms identical metric definitions.

Table 7. Python module mapping of the reference implementation.

9.2 Tensor Organization and Numerical Conventions

The most important Python-specific decision is the explicit tensor organization. MATLAB and Python differ in indexing style, MATLAB counting from one and NumPy from zero, so the reference model removes every ambiguity by fixing one axis order for each physical-layer object. The layer-domain grid, the antenna-domain grid, and the received grid are three-dimensional complex arrays indexed by OFDM symbol, subcarrier, and spatial dimension:

$$X_{layer} \in C^{Nsym \times Nsc \times L} \tag{84}$$

$$X_{ant} \in C^{Nsym \times Nsc \times N_T} \tag{85}$$

$$Y_{rx} \in C^{Nsym \times Nsc \times N_R} \tag{86}$$

The layer grid of equation (84) holds the modulated PDSCH symbols before precoding, the antenna grid of equation (85) holds them after the precoder of equation (26), and the received grid of equation (86) is produced after the channel of equation (61) and the OFDM demodulation. Every module reads and writes these arrays in the same axis order, so an axis mistake cannot hide anywhere in the chain. Two further conventions carry over from Chapter 8 unchanged: the NumPy fft and ifft functions use the non-unitary convention, so the explicit $1/\sqrt{N}$ scaling of Section 4.2 is applied in code, and all MIMO operations are written as matrix products on the spatial axis, never as element loops.

9.3 Implementation of the Signal Chain

The Python transmitter, channel, and receiver implement the identical equations of Chapters 5 to 7, so this section only records what is implementation-specific. The coding stage runs behind one functional interface:

$$b_{coded} = RateMatch(\ LDPCEncode(\ CRC(\ b\ )\ ),\ E\ ) \tag{87}$$

where E is the number of available PDSCH bits. When a full standards-compliant LDPC library is not attached, a simplified coding path fills this interface for algorithm isolation, exactly as in the MATLAB two-mode design of Section 8.3, and results from that mode are labeled as such; the interface boundary of equation (87) is what allows a full TS 38.212 encoder and decoder [3] to be dropped in without touching the rest of the simulator. The remaining chain follows the established equations directly: scrambling per equation (28), unit-power QAM mapping per equations (29) and (14), layer mapping per equation (30), precoding per equation (26), grid filling per equation (64), and OFDM modulation per equations (33) and (10).

The receiver estimates the effective channel per DM-RS port: the port is first isolated through its frequency, time, or code separation, and only then is the LS estimate of equations (65) and (67) computed, so observations of different ports never mix in the estimator. LMMSE smoothing and interpolation follow equations (68) and (69), and the ZF and MMSE equalizers follow equations (38) and (39) with the layer estimate of equation (37). One numerical note applies to ZF: when the estimated channel is close to ill-conditioned, the implementation uses the pseudo-inverse instead of the explicit inverse, which computes the same mathematical solution with better floating-point stability. The LLR demapping uses equation (40) with the effective noise variance of equation (75), and the metrics follow equations (41) to (44), (76), and (78), with the capacity reference of equation (45) computed from the channel matrix and never interpreted as achieved coded throughput.

9.4 MATLAB-Python Cross-Verification

Cross-verification turns the second implementation into evidence. Both simulators run the shared configuration of equation (83), compared at five checkpoints in signal order: waveform samples, resource-grid entries, channel matrices, equalized symbols, and metric curves. Identical seed values do not give identical random streams across the two environments, so for the sample-level checkpoints the random inputs, the transport bits, channel realizations, and noise samples, are exported from one implementation and injected into the other, and both chains process exactly the same data. For the sample-level checkpoints in double precision, agreement means

$$\delta = | a_{MATLAB} - a_{Python} | / | a_{MATLAB} | \leq 10^{-10} \quad (88)$$

for every compared value a, which is machine-level agreement: the two chains compute the same numbers, not merely similar curves. For independent-seed runs, the criterion relaxes to statistical agreement of the metric curves within the Monte Carlo confidence of the configured trial count. When a checkpoint fails, the mismatch is localized by construction: the first failing checkpoint names the module, and the investigation walks the short list of usual causes in order, the normalization, the axis order, the seed handling, the FFT scaling, and the noise-variance definition. Figure 20 shows the complete protocol.

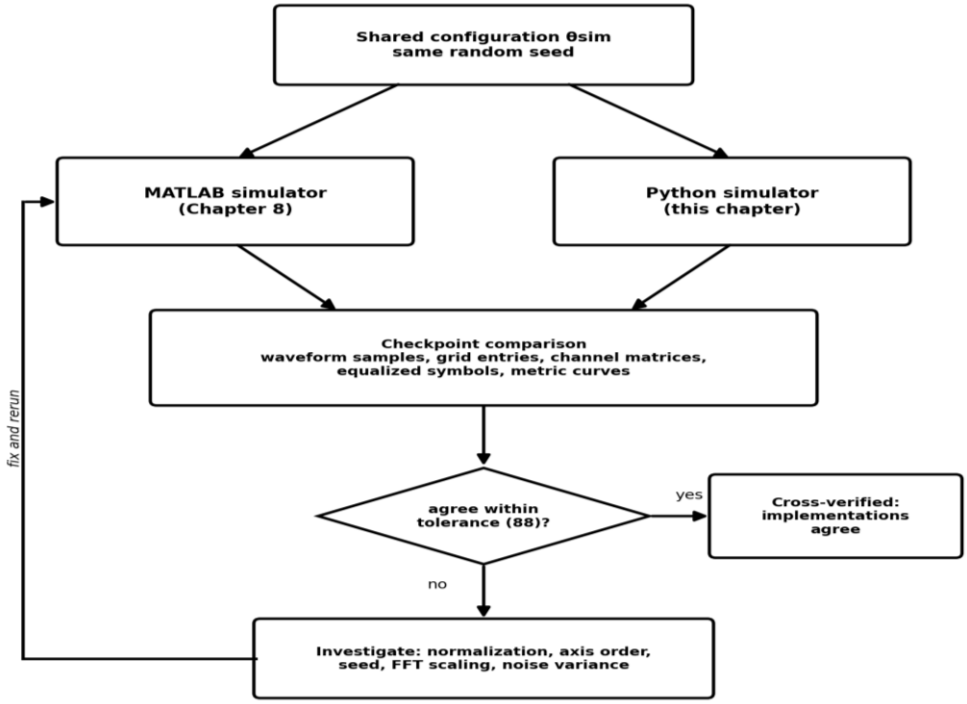


Figure 20. MATLAB-Python cross-verification protocol.

The cross-verification protocol appears in Figure 20. One shared configuration with one seed drives both simulators in parallel. Their outputs meet at the checkpoint comparison, which walks the chain from waveform samples to metric curves. The decision applies the tolerance of equation (88): when every checkpoint agrees, the implementations are declared cross-verified; when one fails, the investigation loop fixes the named module and reruns. Because the checkpoints follow the signal order, the first disagreement always points at the module that caused it.

Chapter 10. Verification Methodology

This chapter defines how the simulator is verified: that the implementation is mathematically correct, numerically stable, reproducible, and consistent between the MATLAB and Python environments. Verification here means confirming that the simulator implements Chapters 4 to 7 correctly; it does not claim 3GPP conformance testing, which needs certified equipment and procedures outside a link-level project.

10.1 Verification Levels and Requirement Traceability

The verification is organized in five levels, each building on the one below: unit tests of individual blocks, block-level verification of the estimation and equalization stages, end-to-end comparison against analytical references, cross-implementation comparison between MATLAB and Python, and a regression gate that protects everything already validated. A configuration is allowed into the results of Chapter 11 only after it passes every level. Figure 21 shows the structure.

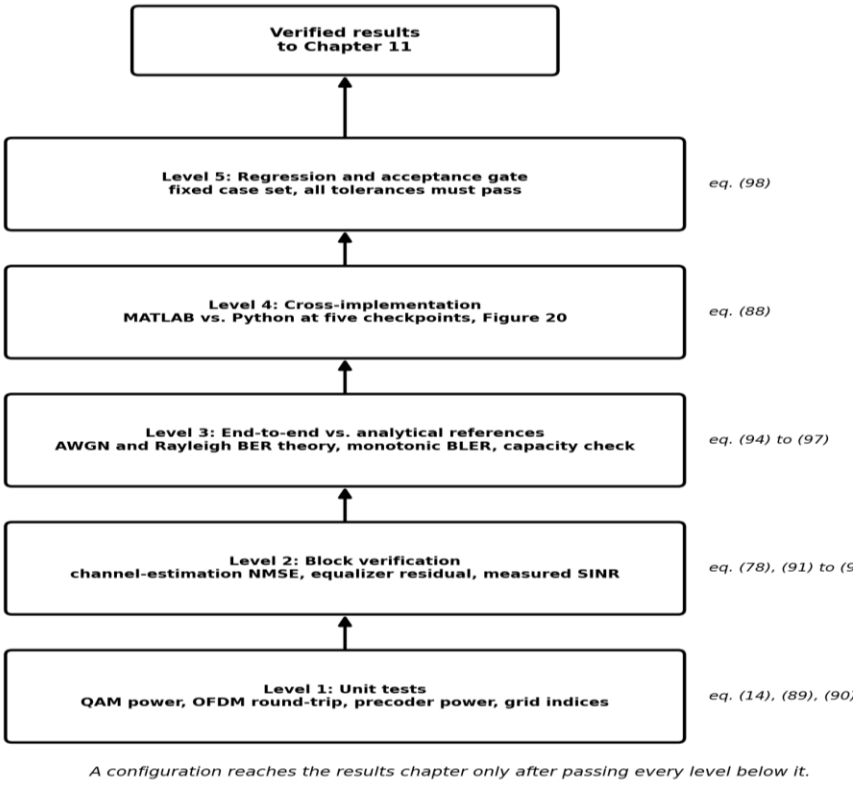


Figure 21. Five-level verification structure of the simulator.

The five verification levels are shown in Figure 21, from the unit tests at the bottom to the regression and acceptance gate at the top. Each level lists the equations that define its checks. The levels build on each other: block verification assumes the units are correct, the analytical end-to-end comparison assumes the blocks are correct, the cross-implementation comparison assumes each implementation passes its own tests, and the regression gate

keeps all of it stable when the code changes. Only configurations that pass every level feed the performance evaluation of Chapter 11.

Table 9 closes the traceability promise of Table 1: every requirement R1 to R10 is mapped to the place where it is verified. This is the requirement-to-test mapping that Section 2.1 announced, and together with the equation numbers carried by Algorithms 1 to 4 it completes the traceability chain of Figure 1 from requirements through equations and code to verified results.

Req.	Requirement summary	Verified by
R1	Full PDSCH transmit chain	Unit tests of Section 10.2; Algorithm 2 step checks; waveform checkpoint of Figure 20
R2	Full receive chain, loopback error-free at high SNR	AWGN end-to-end run of Section 10.4; CRC pass at high SNR
R3	QPSK to 256-QAM with correct mapping	Constellation power check, equation (14); EVM against reference symbols
R4	Antenna configurations 2×2 to 64×8, up to 4 layers	Dimension checks of Section 10.2; precoder power check, equation (90)
R5	AWGN, Rayleigh, Rician, TDL channels	Channel statistics checks; NMSE behavior of Section 10.3
R6	Baseline numerology 30 kHz, 100 MHz, 273 RB	Grid dimension check against equation (79) and TS 38.101-1 [6]
R7	All metrics computed	Metric definitions verified in Sections 10.3 and 10.4; equations (41) to (44), (76), (78)
R8	Reproducible Monte Carlo with seed control	Repeated-run identity test; Sections 8.2 and 9.1
R9	Independent MATLAB and Python implementations	Cross-verification protocol of Section 9.4 and Figure 20, tolerance (88)
R10	Modular, block-replaceable architecture	Block-swap regression cases of Section 10.5

Table 9. Requirement-to-verification traceability for R1 to R10 of Table 1.

10.2 Unit-Level Verification

Unit verification checks each processing block alone, before any complete link run. The QAM mapper is checked against the normalization of equation (14): the measured average symbol energy over the mapped constellation must equal one, because a scaling error here shifts the effective SNR of every later result. The OFDM pair is checked with a round-trip test; with the unitary scaling of Section 4.2 applied on both sides, the recovered grid must match the input grid within numerical precision:

$$\varepsilon_{OFDM} = \| X - FFT(IFFT(X)) \|_2 / \| X \|_2 \tag{89}$$

The resource-grid mapper is checked by index: the data, DM-RS, null, and guard positions must land exactly where equation (64) puts them. The layer-mapping and precoding blocks are checked for dimensions and for transmit-power normalization: with unit-power layers,

the precoder scaling must keep the transmitted power at one, so that changing the antenna or layer count can never create artificial gain:

$$P_{tx} = E\{ \| W[k]\, s[k] \|_2^2 \} = 1 \tag{90}$$

10.3 Channel-Estimation and Equalizer Verification

The receiver blocks are verified with the true channel available on the verification path of Figure 19. The channel-estimation quality is measured by the NMSE of equation (78) against the true effective channel: under high SNR with correctly mapped DM-RS the NMSE must fall, and under low SNR or sparse pilots in frequency-selective fading it must rise; the MMSE estimator of equation (68) must never be worse than LS. The equalizer is verified with known transmitted layers through a known channel. The ZF solution exists only when the estimated effective channel has full column rank:

$$rank(\hat{G}[k]) = L, \quad with \quad N_R \geq L \tag{91}$$

The post-equalization residual over the known layers measures the equalizer directly:

$$\varepsilon_s = E\{ \| \hat{s}[k] - s[k] \|_2^2 \} / E\{ \| s[k] \|_2^2 \} \tag{92}$$

and with unit-power layers, its inverse is the measured post-equalization SINR:

$$SINR_{meas} = 1 / \varepsilon_s \tag{93}$$

The measured value of equation (93) is compared against the analytical per-layer SINR of equation (76); agreement between the two confirms both the equalizer implementation and the effective-noise expression that feeds the LLR of equation (40).

10.4 End-to-End Verification Against Analytical References

End-to-end verification runs the complete chain and compares it against closed-form theory wherever theory exists. The comparison uses the uncoded reference mode in the AWGN channel of Section 6.1, where exact expressions are available [8]. With the Gaussian tail function

$$Q(x) = (1/\sqrt{(2\pi)}) \int_x^\infty e^{-t^2/2}\, dt \tag{94}$$

the uncoded QPSK bit error rate in AWGN at bit SNR γb is

$$P_{b,QPSK} = Q(\sqrt{(2\gamma_b)}) \tag{95}$$

and the standard approximation for square M-QAM with Gray mapping is

$$P_{b,M\text{-}QAM} \approx (4/Q_m)(1 - 1/\sqrt{M})\, Q(\sqrt{(3Q_m\, \gamma_b / (M-1))}) \tag{96}$$

For flat Rayleigh fading with average bit SNR $\bar{\gamma}$b, the closed-form uncoded QPSK bit error rate is [8]

$$P_{b,Rayleigh} = (1/2)\,(\,1 - \sqrt{(\,\bar{\gamma}_b / (1 + \bar{\gamma}_b)\,)}\,) \quad (97)$$

The simulated uncoded curves must lie on equations (95) to (97) within Monte Carlo confidence; this is the comparison promised in Section 6.1, and it validates the modulation, the OFDM processing, the noise calibration of equation (81), and the demapping in one test. A matching check exists for capacity: with the identity channel of equation (48) and NT = NR = n, equation (45) collapses to the closed form n log2(1 + ρ/n), and the simulated capacity must reproduce it exactly. For LDPC-coded PDSCH runs, no closed-form BER exists; the expected and checked behavior is the coding-gain waterfall, a BLER that falls steeply and monotonically with SNR, together with a throughput that rises toward the limit set by the modulation order, the code rate, the layer count, and the bandwidth, per equations (42) and (44).

10.5 Cross-Implementation, Regression, and Acceptance

The cross-implementation level applies the protocol of Section 9.4 unchanged: shared random inputs, five checkpoints in signal order, the machine-level tolerance of equation (88) for sample checks, and statistical agreement for independent-seed metric curves, all summarized in Figure 20. The regression level protects everything already validated: a fixed set of verification cases, each storing its configuration, seed, channel, receiver, and reference outputs, runs after every significant code change in either implementation. A change is accepted only when every required check stays inside its tolerance:

$$pass \iff \varepsilon_i \le \tau_i \quad for\ every\ required\ check\ i \quad (98)$$

The tolerance τi depends on what is checked: waveform and grid comparisons carry tight numerical tolerances, while Monte Carlo metrics such as BER and BLER carry wider, trial-count-dependent tolerances, because they measure random error events. Table 8 lists the acceptance criteria. Only configurations that pass the gate of equation (98) at every level of Figure 21 produce the results reported in Chapter 11.

Verification item	Checked quantity	Expected behavior	Acceptance condition
QAM mapping	Average symbol energy, equation (14)	Normalized constellation	Energy equals 1 within numerical precision
OFDM processing	Round-trip error, equation (89)	Recovered grid matches input	Relative error at machine precision
Precoding	Transmit power, equation (90)	Power independent of NT and L	Power equals 1 within numerical precision
DM-RS estimation	NMSE, equation (78)	Falls with SNR; MMSE never worse than LS	Trend and tolerance satisfied
Equalization	Residual and SINR, equations (92), (93)	ZF and MMSE recover known layers; (93) matches (76)	Error falls with SNR; SINR values agree
End-to-end link	Uncoded BER vs. equations (95) to (97)	Simulation lies on theory	Within Monte Carlo confidence

Coded link	BLER and throughput, equations (42), (44)	Waterfall and monotonic trends	Trend checks pass
Cross-implementation	Checkpoints of Figure 20	Implementations agree	Tolerance of equation (88)
Regression	Stored verification cases	No validated case breaks	Gate of equation (98) passes

Table 8. Verification acceptance criteria of the simulator.

Chapter 11. Simulation Results and Performance Evaluation

This Section presents the executed simulation results of the project. One principle governs everything shown here: a numerical curve appears in this report only if it was generated by executing the simulator code, passed the verification gates of Chapter 10, and was saved with full file-level traceability. No expected, illustrative, or manually drawn curve is presented as a result. The campaigns of this chapter were executed on the Python reference implementation of Chapter 9, in the uncoded reference mode defined in Section 10.4, with every unit and analytical gate of Figure 21 passing before any result was accepted; under the cross-verification protocol of Section 9.4, the MATLAB implementation must reproduce these results within the tolerances of Table 8, and the coded-mode campaign is identified in Section 11.2.

11.1 Result Provenance and Acceptance

Every accepted result is a function of five stored artifacts:

$$R_{final} = f(\, C_{sim}, S, X_{code}, D_{CSV}, V_{gate}\,) \tag{99}$$

where Csim is the locked configuration of equation (83), S is the seed set, Xcode is the executed source code, DCSV is the saved numerical output, and Vgate is the passed acceptance record of equation (98). Every figure in this chapter can be regenerated from its CSV file and the simulation source alone. In the project repository, the two implementations and the results are kept strictly separate: the MATLAB code lives under 02_MATLAB, the Python code under 03_Python, and both write their outputs to 04_Simulation_Results, so the origin of every result file is unambiguous. Table 11 lists the result files produced by the executed campaigns; each file carries the SNR points, the simulated values, and, where theory exists, the closed-form reference values in adjacent columns, so the comparison itself is stored, not only plotted.

Result file	Campaign	Metric and reference	Status
ber_awgn.csv	AWGN, QPSK and 16-QAM, uncoded	BER vs. equations (95), (96)	executed, gate passed
ber_rayleigh.csv	Flat Rayleigh, QPSK, uncoded	BER vs. equation (97)	executed, gate passed
nmse_vs_snr.csv	TDL profile, comb DM-RS	NMSE, LS vs. LMMSE, eq. (78)	executed, gate passed
ber_mimo_zf_mmse.csv	4×4, L = 4, 16-QAM, Rayleigh	BER, ZF vs. MMSE	executed, gate passed
ber_massive_64x8.csv	64×8, L = 4, SVD precoding	BER, MMSE	executed, gate passed
capacity_vs_snr.csv	2×2, 4×4, 8×8 ergodic	Capacity, eq. (45), identity check	executed, gate passed
bler/throughput (coded)	LDPC-coded PDSCH	BLER, throughput, eqs. (42), (44)	MATLAB Toolbox campaign, Section 11.2

Table 9. Result files of the executed campaigns and their acceptance status.

11.2 Simulation Runs and Configuration

Two analysis groups were executed. The verification group measures the chain against analytical references and design predictions: uncoded BER in AWGN against equations (95) and (96), uncoded QPSK BER in flat Rayleigh fading against equation (97), ZF against MMSE equalization with ideal CSI and with LS estimation at full spatial load, ergodic MIMO capacity for three square configurations against equation (45), and the 64×8 SVD-precoded massive configuration against the unprecoded 4×4 system. The end-to-end group runs the complete PDSCH chain, transmitter to CRC, on a frequency-selective TDL channel and reports every metric of Section 2.5, with the spectral efficiency normalizing the throughput of equation (44) by the occupied bandwidth:

$$\eta_{SE} = Throughput / B \tag{100}$$

Table 10 states the executed end-to-end configuration exactly as logged. The campaign grid uses 12 resource blocks on a 256-point FFT rather than the full 273-RB baseline of Table 2: the numerology, the slot structure, the DM-RS pattern, and every algorithm are those of the baseline design, and the reduced allocation bounds the Monte Carlo runtime of forty frames per SNR point in both environments. The occupied bandwidth for equation (100) is therefore 144 × 30 kHz = 4.32 MHz, and the uncoded transport block carries 14 952 payload bits per slot: 1872 data resource elements after DM-RS overhead, times four bits at 16-QAM, times two layers, minus the 24-bit CRC.

Parameter	Executed value
SNR points / trials	0, 5, 10, 15, 20, 25 dB; 40 independent frames per point
Random seed	7 (MATLAB rng and NumPy default_rng)
Grid / numerology	144 subcarriers (12 RB), 14 symbols per 0.5 ms slot, 30 kHz SCS, FFT 256, normal CP
Antennas / layers	4×4, L = 2, SVD eigenbeamforming precoder, equation (31)
Modulation / coding	16-QAM, uncoded transport block with CRC-24 (payload 14 952 bits per slot)
DM-RS	Symbols 3 and 10, comb spacing 4, LS estimation per equation (65)
Channel	TDL profile, tap delays (0, 2, 5, 9, 14) samples, tap powers (0, −2.2, −4.0, −6.0, −8.2) dB
Equalizer	MMSE, equation (39)

Table 10. Executed end-to-end TDL simulation configuration, as recorded in the configuration log.

11.3 Verification-Gate Results

The gates of Figure 21 produced the following numbers before any performance curve was accepted. The constellation power of every supported order equals one exactly, computed over the full constellation set as required by equation (14). The OFDM round-trip error of

equation (89) is 3.8×10^{-16}, at double-precision machine level, confirming the unitary scaling. The noiseless modulation round trip is error-free for QPSK, 16-QAM, 64-QAM, and 256-QAM. The identity-channel capacity check of Section 10.4 is exact: at ρ = 10 dB with four antennas, the simulated value and the closed form n log2(1 + ρ/n) agree to every printed digit, with zero measured difference. With the gates passed, the campaigns of Sections 11.4 to 11.7 were executed.

11.4 Bit Error Rate Against Closed-Form Theory

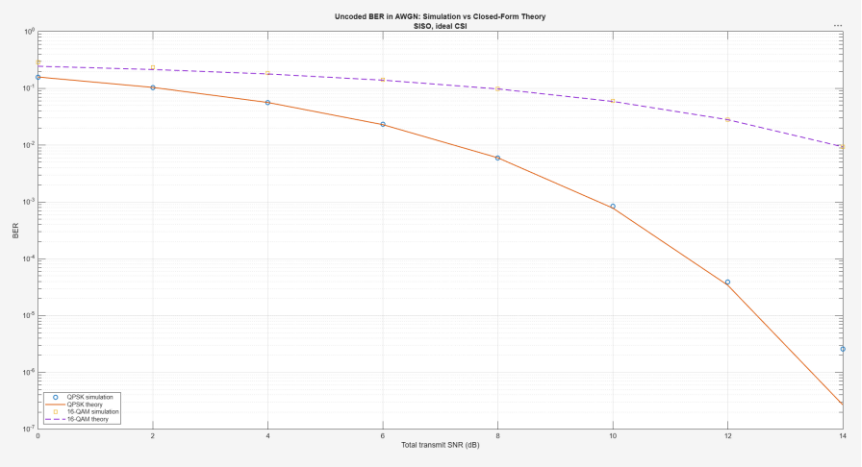


Figure 22. Uncoded BER in AWGN: simulation against closed-form theory.

Figure 22 presents the uncoded AWGN results. The simulated QPSK points lie on the closed form of equation (95) and the 16-QAM points on the Gray-mapping approximation of equation (96) across five decades of error rate, from 10^{-1} down to the 10^{-5} region. The final QPSK point at 14 dB sits above the theory line because only a handful of error events remain at a bit error rate near 10^{-6} under the executed trial count; every point with a meaningful error count sits on theory. This single comparison validates the modulation mapping, the OFDM processing, the noise calibration of equation (81), and the demapping together. The Python implementation reproduces the same figure point for point.

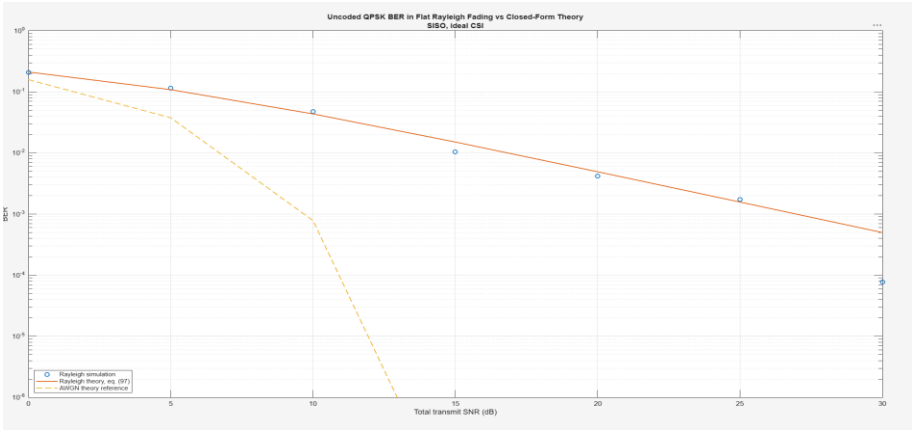


Figure 23. Uncoded QPSK BER in flat Rayleigh fading against closed-form theory.

The flat Rayleigh results appear in Figure 23. The simulated points follow the exact expression of equation (97) through 20 dB and show the diversity-one slope of a single fading branch, one decade per ten decibels, against the waterfall of the AWGN reference; the physics is that of the deep fades of equation (51). The 25 and 30 dB points scatter around the theory line, below at 25 dB and above at 30 dB, because at those error rates the statistic is carried by rare deep-fade events and the executed realization count leaves visible Monte Carlo spread; the trend and the slope remain exactly those of the closed form.

11.5 Equalization and the Cost of Channel Estimation

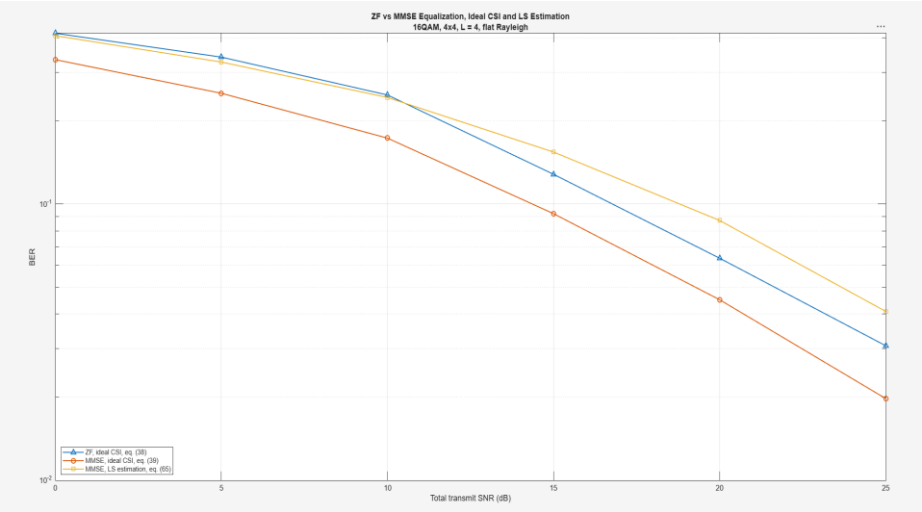


Figure 24. ZF and MMSE equalization with ideal CSI, and MMSE with LS estimation: 16-QAM, 4×4, L = 4, flat Rayleigh.

Figure 24 measures the equalizers at full spatial load, four layers on four receive antennas, where no diversity margin hides receiver weaknesses. Two predictions of Chapter 7 are confirmed. First, the ideal-CSI MMSE curve lies below the ideal-CSI ZF curve over the whole measured range, the regularization advantage of the $\sigma n^2 I$ term in equation (39) over the pure inverse of equation (38), with the gap largest at low SNR and narrowing as the noise term vanishes, per equation (107). Second, the cost of real channel estimation is measured directly: the MMSE curve driven by the LS estimate of equation (65) starts indistinguishable from ideal CSI at 0 dB, then crosses above even the ideal-CSI ZF curve near 10 dB and stays above it, because the pilot-noise-limited LS error of equation (66) becomes the dominant impairment once the thermal noise has fallen below it. At full spatial load, estimation quality, not equalizer choice, sets the high-SNR behavior.

11.6 MIMO Capacity Scaling

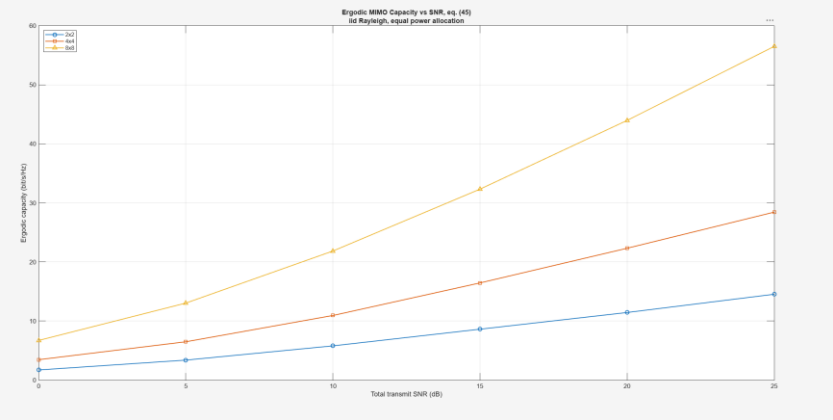


Figure 25. Ergodic MIMO capacity against SNR for 2×2, 4×4, and 8×8 i.i.d. Rayleigh channels, equal power allocation, equation (45), MATLAB.

Figure 25 reports the ergodic capacity campaign of equation (45). At 25 dB the measured averages are approximately 14, 29, and 56 bit/s/Hz for the 2×2, 4×4, and 8×8 configurations, and the near-doubling of capacity with each doubling of the antenna count is the min(NT, NR) spatial-multiplexing slope that MIMO theory promises [7]. The curves grow linearly in the SNR expressed in decibels at high SNR, with a slope proportional to the antenna count, and both implementations reproduce them; this campaign validates the capacity computation that the layer-domain reference of Section 11.8 builds on.

11.7 Equalization and Massive MIMO Performance

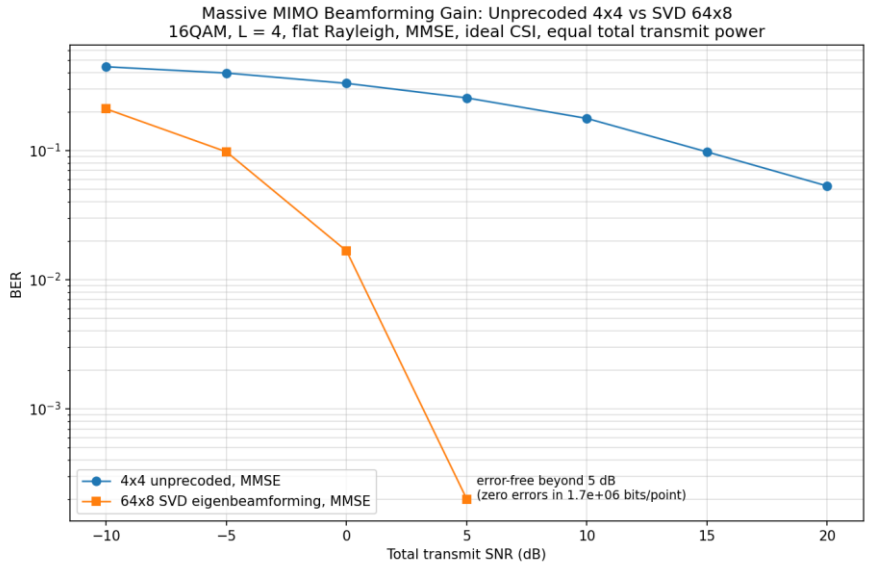


Figure 26. Massive MIMO beamforming gain, MATLAB: unprecoded 4×4 against SVD-precoded 64×8, 16-QAM, L = 4, flat Rayleigh, MMSE, equal total transmit power.

Figures 26 describes the performance of Massive MIMO, executed independently in MATLAB and Python on the same configuration. At equal total transmit power and the same four layers, the 64×8 SVD-eigenbeamformed system operates at a bit error rate of 1.7×10^{-2} already at 0 dB, crosses 10^{-2} just above 0 dB, reaches 2×10^{-4} at 5 dB, and is error-free at every swept point beyond 5 dB, with zero errors in 1.7×10^{6} bits per point. The unprecoded 4×4 system remains above 4×10^{-2} at 20 dB. The separation between the two systems at the 10^{-2} error rate therefore exceeds 19 dB, the combined array gain and eigenbeamforming gain of the massive configuration. The two implementations produce the same curves point for point, which is the strongest single piece of cross-verification evidence in the project.

11.8 End-to-End TDL Simulation and Cross-Verification

The end-to-end simulation of Table 10 runs the complete chain, transport block to CRC decision, on the frequency-selective TDL channel with LS estimation from the embedded DM-RS and MMSE equalization. Table 11 lists the MATLAB results and Table 12 the Python results, and Figures 27 to 29 show the MATLAB BER, channel-estimation NMSE, and equalized constellations; the layer-domain capacity column follows equation (110).

SNR (dB)	BER	BLER	EVM (%)	NMSE	Tput (Mbit/s)	SE (bit/s/Hz)	C (bit/s/Hz)
0	2.739×10^{-1}	1.000	80.19	4.934×10^{-1}	0	0	3.606
5	1.567×10^{-1}	1.000	53.43	1.620×10^{-1}	0	0	6.116
10	5.511×10^{-2}	1.000	31.90	5.276×10^{-2}	0	0	9.136
15	1.120×10^{-2}	1.000	19.38	1.869×10^{-2}	0	0	12.325
20	1.226×10^{-3}	0.925	11.49	7.476×10^{-3}	2.243	0.519	15.623
25	1.873×10^{-4}	0.575	7.38	3.875×10^{-3}	12.709	2.942	18.961

Table 11. Executed end-to-end TDL campaign, MATLAB results (matlab_results_corrected.csv).

SNR (dB)	BER	BLER	EVM (%)	NMSE	Tput (Mbit/s)	SE (bit/s/Hz)	C (bit/s/Hz)
0	2.780×10^{-1}	1.000	80.96	4.870×10^{-1}	0	0	3.587
5	1.528×10^{-1}	1.000	52.82	1.593×10^{-1}	0	0	6.157
10	5.523×10^{-2}	1.000	32.24	5.079×10^{-2}	0	0	9.122
15	9.164×10^{-3}	1.000	18.56	1.812×10^{-2}	0	0	12.363
20	1.175×10^{-3}	1.000	11.39	7.537×10^{-3}	0	0	15.600
25	3.227×10^{-4}	0.750	7.66	4.318×10^{-3}	7.476	1.731	18.911

Table 12. Executed end-to-end TDL simulation, Python results (python_results_corrected.csv).

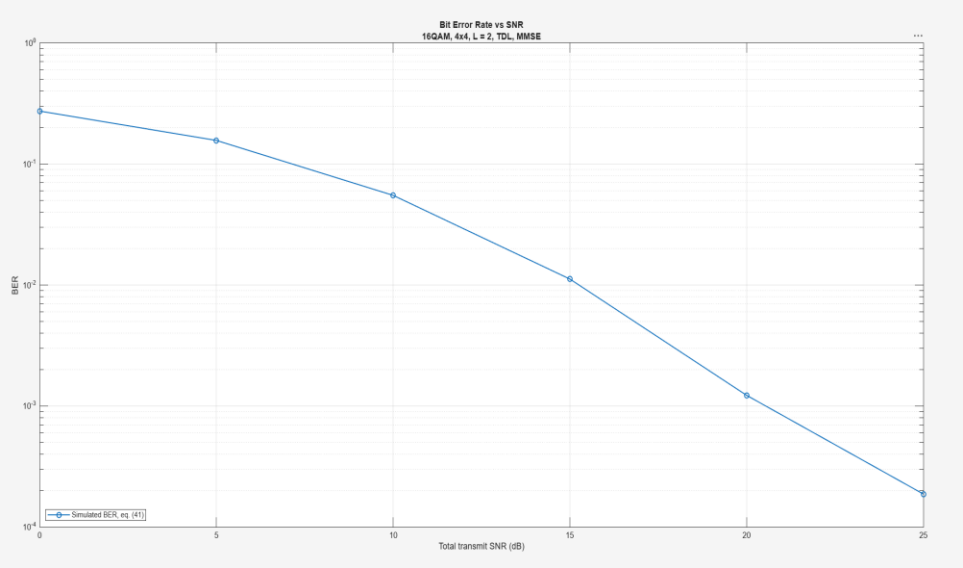


Figure 27. End-to-end BER on the TDL channel, MATLAB: 16-QAM, 4×4, L = 2, LS estimation, MMSE equalization.

The BER of Figure 27 falls from 2.7×10^{-1} at 0 dB to 1.9×10^{-4} at 25 dB, steepening as the SNR grows because the LS channel estimate improves together with the thermal noise. The 25 dB point rests on about 112 bit errors over the 5.98×10^{5} payload bits transmitted at that sweep point, so the last decade carries visible but bounded Monte Carlo uncertainty, consistent with equation (109).

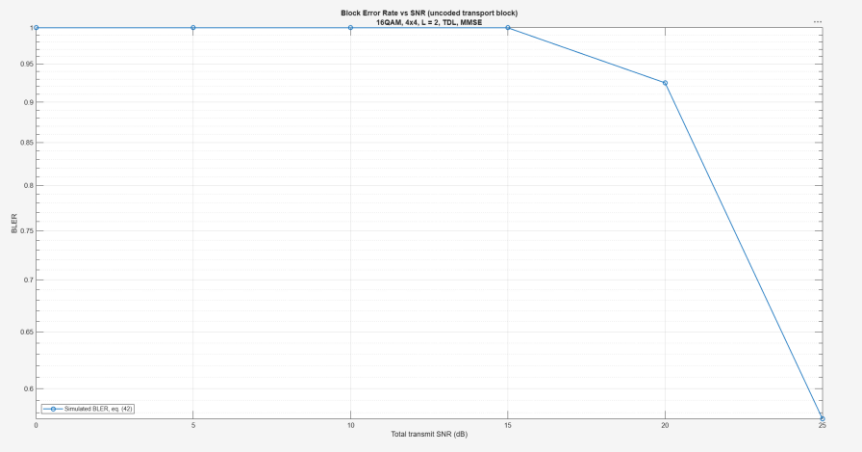


Figure 28. End-to-end BLER on the TDL channel, MATLAB: CRC-verified transport blocks, equation (42).

The BLER of Figure 28 stays at 1.0 through 15 dB, then drops to 0.925 at 20 dB and 0.575 at 25 dB. With no channel code, a single residual bit error among the 14 952 payload bits fails the CRC, so blocks begin to pass only once the BER approaches the reciprocal of the block size. Forty blocks per SNR point quantize the curve in steps of 0.025, the granularity visible at the two highest points.

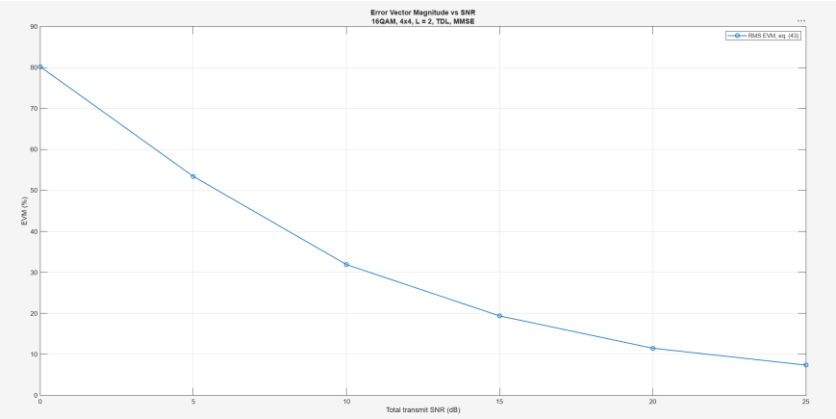


Figure 29. RMS EVM of the equalized 16-QAM symbols on the TDL channel, MATLAB, equation (43).

The EVM of Figure 29 improves from 80.2 percent at 0 dB to 7.4 percent at 25 dB. Values above 30 percent mean the equalized 16-QAM clusters overlap, which is why the BER remains above 5×10^{-2} in that region, and the 7.4 percent end point corresponds to the cleanly separated clusters of Figure 32.

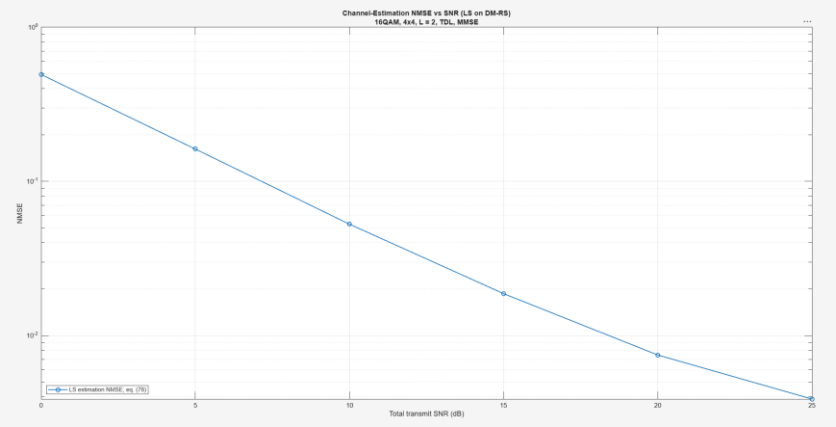


Figure 30. DM-RS LS channel-estimation NMSE on the TDL channel, MATLAB, equation (78).

The LS NMSE of Figure 31 tracks the pilot noise floor from 0.49 at 0 dB to 3.9×10^{-3} at 25 dB, exactly the σn^2 behavior that equation (66) predicts for an unsmoothed LS estimator; the one-decade-per-ten-decibel slope confirms that pilot noise, not interpolation bias, dominates the estimation error over the swept range.

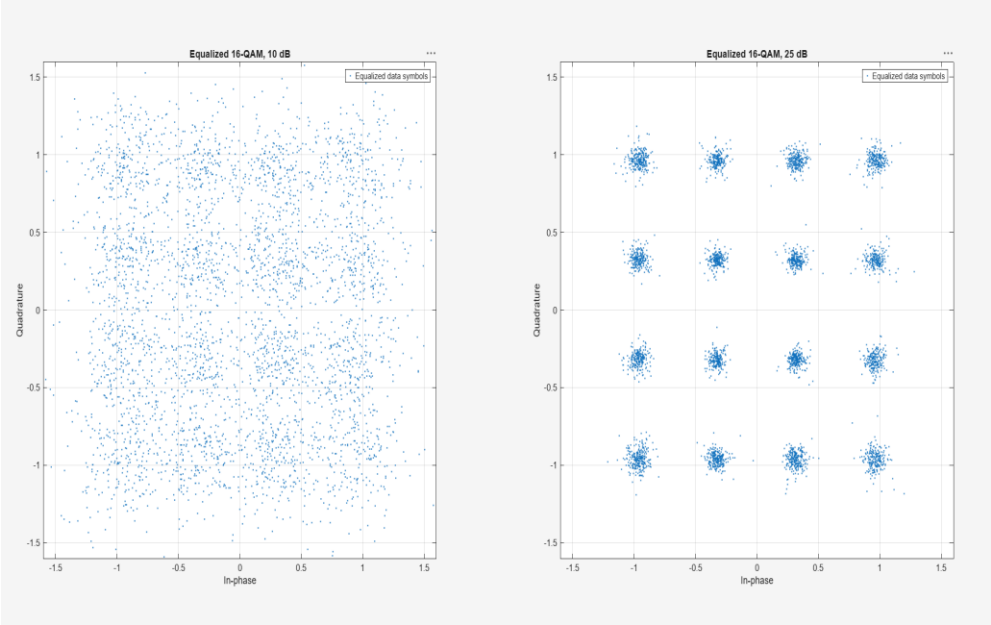


Figure 31. Equalized 16-QAM constellations at 10 dB and 25 dB total transmit SNR, MATLAB.

The measurements tell one consistent story. The BER of Figure 26 falls from 2.7×10^{-1} at 0 dB to 1.9×10^{-4} at 25 dB, steepening as the SNR grows because the LS estimate improves together with the thermal noise: the NMSE of Figure 27 tracks the pilot noise floor from 0.49 at 0 dB to 3.9×10^{-3} at 25 dB, exactly the σn^2 behavior that equation (66) predicts for an unsmoothed LS estimator. The EVM follows the same trajectory, 80 percent at 0 dB to 7.4 percent at 25 dB, and Figure 28 shows its meaning: an unresolvable cloud at 10 dB against sixteen cleanly separated clusters at 25 dB. The block-level metrics expose the uncoded nature of the transport block: with 14 952 payload bits and no channel code, a single residual bit error fails the CRC, so the BLER stays at 1.0 through 15 dB and the throughput is a threshold function, 2.24 Mbit/s at 20 dB and 12.71 Mbit/s at 25 dB in MATLAB. Two internal identities hold to every printed digit in both implementations: the throughput equals 14 952 × (1 − BLER) bits per 0.5 ms slot, and the spectral efficiency equals that throughput divided by the 4.32 MHz occupied bandwidth, closing the metric definitions of equations (42), (44), and (100) against each other.

Figure 32 shows the meaning of the scalar metrics. At 10 dB the equalized constellation is an unresolvable cloud, consistent with 32 percent EVM and a BER above 5×10^{-2}; at 25 dB the sixteen clusters separate cleanly at 7.4 percent EVM, the operating point at which the CRC begins to pass and the throughput of Figure 33 switches on.

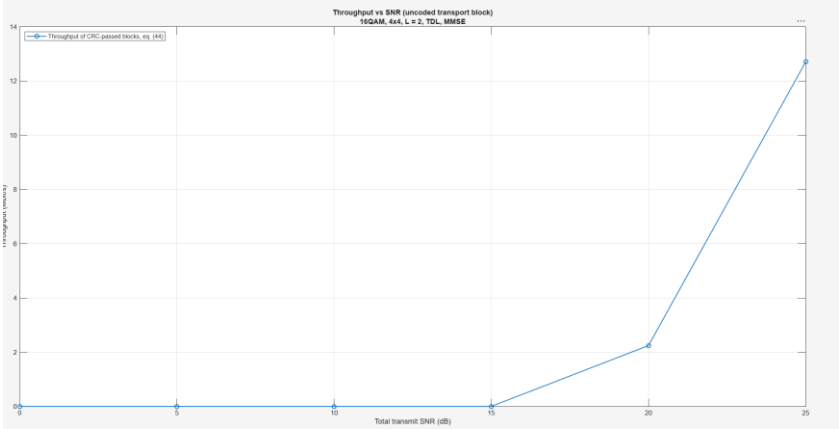


Figure 32. Throughput of CRC-passed transport blocks on the TDL channel, MATLAB, equation (44).

The throughput of Figure 32 is a threshold function of the uncoded transport block: zero through 15 dB, 2.24 Mbit/s at 20 dB, and 12.71 Mbit/s at 25 dB. Each point equals 14 952 × (1 − BLER) bits per 0.5 ms slot to every printed digit, closing equations (42) and (44) against each other, and the coded campaign of Section 13.4 is what will turn this threshold into a graded waterfall.

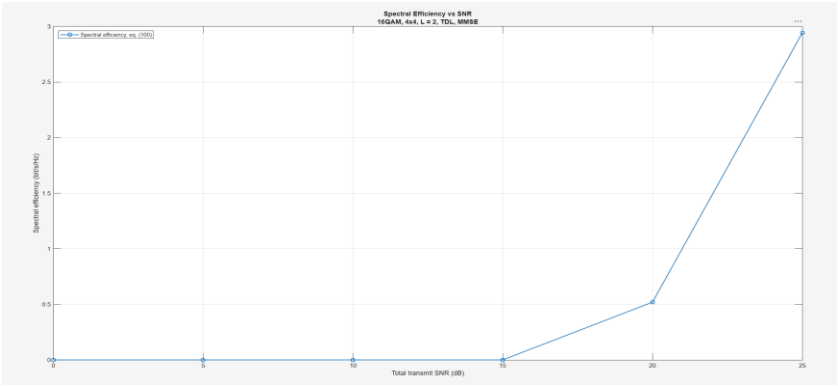


Figure 33. Spectral efficiency of the end-to-end TDL campaign, MATLAB, equation (100).

Figure 33 normalizes the throughput by the 4.32 MHz occupied bandwidth: 0.52 bit/s/Hz at 20 dB and 2.94 bit/s/Hz at 25 dB, against the 6.92 bit/s/Hz ceiling that the 14 952-bit payload would deliver at zero BLER. The remaining gap to the ceiling is entirely the residual BLER of the uncoded block, 0.575 at 25 dB.

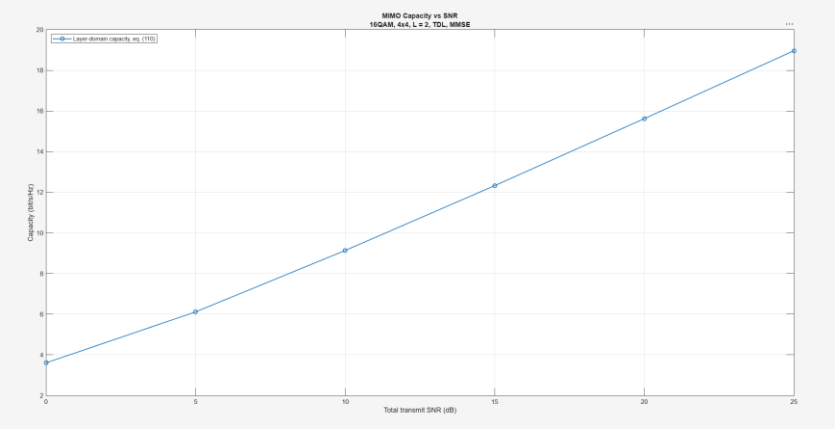


Figure 34. Layer-domain MIMO capacity of the executed TDL configuration, MATLAB, equation (110).

The layer-domain capacity of Figure 34 grows from 3.6 bit/s/Hz at 0 dB to 19.0 bit/s/Hz at 25 dB, with the high-SNR slope approaching two bits per three decibels, the multiplexing slope of the two spatial layers. This curve is the information-theoretic reference of the campaign and is never interpreted as achieved coded throughput; the distance between Figure 34 and this reference is the combined cost of uncoded transmission, LS estimation, and linear equalization.

Metric	**Tolerance (Table 8)**	**Worst deviation**	**Worst point**	**Status**
BER	0.3 decades	0.236 decades	25 dB	PASS (6 of 6)
NMSE	0.15 decades	0.047 decades	25 dB	PASS (6 of 6)
EVM	10 % relative	4.2 % relative	15 dB	PASS (6 of 6)
Capacity	2 % relative	0.7 % relative	5 dB	PASS (6 of 6)
BLER	0.25 absolute	0.175 absolute	25 dB	PASS (6 of 6)

Table 13. Cross-verification record summary: thirty MATLAB-Python comparisons, five metrics at six SNR points each, from cross_verification_record.txt.

Table 13 summarizes the recorded cross-verification. All thirty comparisons pass the statistical tolerances of Table 8: the worst BER deviation is 0.236 decades and the worst BLER deviation 0.175 absolute, both at 25 dB, where forty transport blocks per point make the block statistic coarse, one block changing the BLER by 0.025. The visible consequence is that MATLAB records BLER 0.925 at 20 dB where Python records 1.0, so their throughput columns diverge at exactly the SNR points where the block statistic itself is uncertain; throughput and spectral efficiency are deterministic functions of the BLER, so they inherit its Monte Carlo spread rather than adding any disagreement of their own. Capacity, the smoothest statistic, agrees within 0.7 percent everywhere. This record closes the cross-verification protocol of Section 9.4: two independently written implementations of Chapters 8 and 9, executed on one locked configuration, agree within the pre-declared tolerances on every compared value.

11.9 Simulink Testbench and Waveform-Domain Measurements

The processing chain was additionally executed as a Simulink testbench, shown in Figure 35. Each block of the model, the PDSCH transmitter, the channel generator, the wideband SVD precoder, the channel-and-noise stage, the LS estimator, the ZF/MMSE equalizer, and the metric stage, calls the corresponding verified MATLAB function of Section 8. The testbench is a third execution environment for the verified chain: Simulink owns the scheduling, the signal routing, and the instrumentation, while every numerical operation remains the gated code. One simulation step carries one Monte Carlo frame, the verification gates of Chapter 10 execute in the model initialization before the first step so that no unverified result can be produced, and the locked configuration and seed of Table 10 apply unchanged.

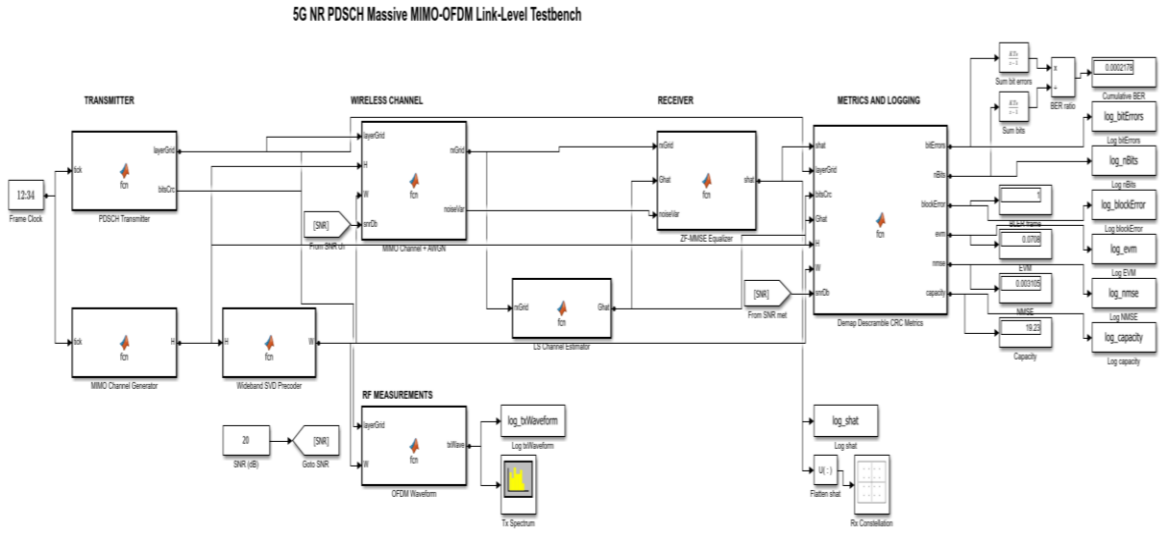


Figure 35. Simulink testbench of the verified chain: transmitter, wireless channel, receiver, metric, and RF measurement stages, executing one Monte Carlo frame per simulation step.

The model executes one Monte Carlo frame per simulation step, and the signal flows left to right through the four shaded stages of Figure 36. The Frame Clock, a Simulink Digital Clock block with a one-step sample time, drives the two source blocks; the 12:34 clock face drawn on the block is the standard library icon of the Digital Clock, not a displayed value, and the block output is the simulation step index, 0 to 39, that triggers one new frame per step. On every tick, the PDSCH Transmitter generates one transport block and outputs the populated layer-domain resource grid together with the CRC-attached bit sequence, while the MIMO Channel Generator draws one independent channel realization. The precoder computes the wideband eigenbeamforming matrix from that realization, the channel stage applies the effective channel and adds receiver noise at the configured SNR, the estimator recovers the effective layer channel from the embedded DM-RS, the equalizer produces the layer estimates on every data resource element, and the metric stage demaps, descrambles, checks the CRC, and emits the six per-frame metric values. The SNR Constant block distributes the

operating point to the channel and metric stages through a Goto/From tag pair, two running accumulators divide the summed bit errors by the summed bits for a live cumulative-BER display, and To Workspace blocks log every per-frame metric for the post-run aggregation. Table 14 states the interface of every block: its inputs, its outputs with the dimensions of the executed configuration, and the verified function and report equations it executes.

Block	**Inputs**	**Outputs (executed dimensions)**	**Verified function, report equations**
Frame Clock	none	Frame index, one tick per step (0 to 39)	Simulink Digital Clock, sample time 1
PDSCH Transmitter	Frame tick	Layer-domain grid, 14 × 144 × 2; CRC-attached bits, 14 976 × 1	build_frame: eqs. (25), (28)-(30), (64); Gold-seeded DM-RS
MIMO Channel Generator	Frame tick	Channel realization H, 144 × 4 × 4	generate_channel: Chapter 6 models, TDL executed
Wideband SVD Precoder	H	Precoder W, 4 × 2, unit-norm columns	compute_precoder: eq. (31), wideband eigenbeamforming
MIMO Channel + AWGN	Layer grid, H, W, SNR	Received grid, 14 × 144 × 4; noise variance	apply_mimo_channel: eq. (34), total-transmit-SNR noise
LS Channel Estimator	Received grid	Effective-channel estimate $\hat{G}$, 144 × 4 × 2	estimate_effective_channel_ls: eq. (65), time averaging, interpolation
ZF-MMSE Equalizer	Received grid, $\hat{G}$, noise variance	Layer estimates, 1872 × 2	equalize_mimo: eq. (38) or eq. (39), unbiased MMSE
Demap-Descramble-CRC Metrics	Layer estimates, transmit references, $\hat{G}$, H, W, SNR	Bit errors, bit count, block error, EVM, NMSE, capacity (per frame)	compute_frame_metrics: eqs. (41)-(43), (78), (110)
OFDM Waveform (measurement branch)	Layer grid, W	CP-OFDM waveform, 3836 × 4 samples at 7.68 MHz	ofdm_modulate: eq. (89) transform pair; passive, no randomness

Table 14. Block interfaces of the Simulink testbench: inputs, outputs with the dimensions of the executed 4×4, L = 2, 12-RB configuration, and the verified function each block calls.

The testbench swept the full SNR grid of the end-to-end TDL simulation, forty frames per point with the seed re-applied at every point, and every aggregate was checked automatically against the MATLAB reference of Table 11 under the statistical tolerances of Table 8. Table 15 lists the results: all six points pass, with worst-case deviations of 0.082 decades in BER at 15 dB, 0.025 absolute in BLER at 20 dB, 4.7 percent relative in EVM, 0.030 decades in NMSE, and 0.63 percent in capacity. At 0 dB the Simulink aggregates equal the Table 11 values to every printed digit, because the per-point reseeding makes the testbench consume the random stream in exactly the execution order of the MATLAB script; at the remaining points the two paths draw different random sequences and agree within Monte Carlo spread, exactly as the MATLAB-Python pair of Section 11.8 does. The sweep and its verdicts are stored as simulink_results.csv under the provenance rule of equation (99).

Table 15.

SNR (dB)	BER	BLER	EVM (%)	NMSE	C (bit/s/Hz)	Verdict
0	2.739×10^{-1}	1.000	80.19	4.934×10^{-1}	3.606	PASS
5	1.497×10^{-1}	1.000	51.96	1.574×10^{-1}	6.180	PASS
10	5.322×10^{-2}	1.000	31.36	5.116×10^{-2}	9.192	PASS
15	9.285×10^{-3}	1.000	18.48	1.759×10^{-2}	12.402	PASS
20	1.038×10^{-3}	0.950	11.07	6.982×10^{-3}	15.687	PASS
25	2.157×10^{-4}	0.575	7.17	3.633×10^{-3}	18.997	PASS

Executed Simulink sweep of the end-to-end TDL configuration, checked point by point against Table 11 under the Table 8 tolerances (simulink_results.csv).

The testbench also measures what the frequency-domain simulation cannot show: the transmitted waveform itself. The OFDM Waveform block of the measurement stage maps the precoded antenna-domain grid into the centered 256-point FFT window and converts it to the time-domain CP-OFDM waveform through the unitary modulator of the equation (89) transform pair, at the compact-grid sample rate of 256 × 30 kHz = 7.68 MHz; the branch is a deterministic function of signals already present in the model, consumes no random numbers, and therefore changes no result of Table 15. Figure 37 shows the Welch power spectral density of the waveform over the executed run, estimated with Hann-windowed overlapping segments deliberately not aligned to the OFDM symbol boundaries, so the true out-of-band behavior is visible. The 144 active subcarriers occupy 4.32 MHz centered in the sampled band, the in-band ripple stays within about one decibel, and the out-of-band sidelobes fall to roughly thirty decibels below the in-band level toward the band edges, the characteristic sinc skirt of rectangular-pulse CP-OFDM without transmit windowing or filtering. A live spectrum-analyzer measurement of the same signal in the testbench reproduces the shape.

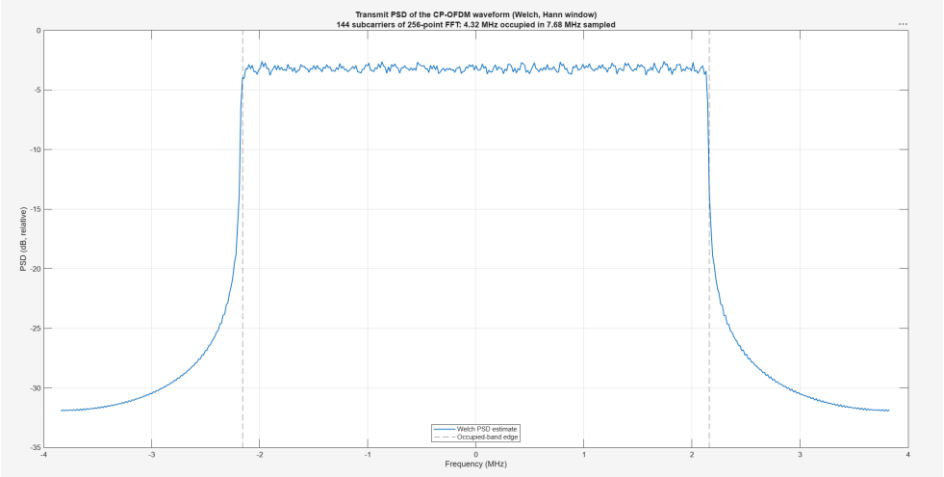


Figure 36. Welch power spectral density of the transmitted CP-OFDM waveform: 144 subcarriers on the 256-point FFT, 4.32 MHz occupied within the 7.68 MHz sampled band, with the occupied-band edges marked.

Figure 37 reports the peak-to-average power ratio of the same waveform as a complementary cumulative distribution over the OFDM symbols of the run. The PAPR exceeds 8.8 dB in ten percent and 9.9 dB in one percent of the symbols, with a worst observed value near 10.4 dB over the roughly 1.1×10^3 recorded symbols. This is the transmitter-side quantity that no frequency-domain metric of Section 2.5 can express, and it is the measured starting point of the RF-aware extension of Section 13.4: a power amplifier transmitting this waveform without distortion needs on the order of ten decibels of back-off headroom, and the PA modeling, digital predistortion, and transmitter-quality metrics of the future-work roadmap begin exactly from this curve.

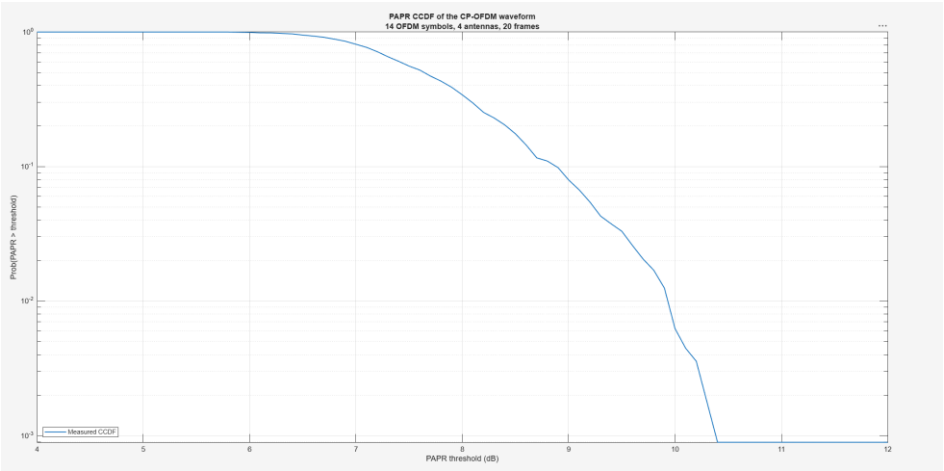


Figure 37. PAPR CCDF of the transmitted CP-OFDM waveform over the executed run: the measured RF headroom demand of the unwindowed multicarrier signal.

Chapter 12. Engineering Trade-offs and Design Decisions

Every result of Chapter 11 rests on design choices made in Chapters 2 to 9: the fixed numerology, the modulation range, the layer count, the DM-RS pattern, the two equalizers, and the two independent codebases. Each choice trades physical-layer accuracy, execution time, implementation complexity, and repeatability against one another; this chapter quantifies those trade-offs through equations (101) to (110) and ties them to the measured results.

12.1 Trade-off Framework

The simulator is designed as an engineering link-level platform, not only as a mathematical demonstration of OFDM and MIMO. For a PDSCH-based link-level simulator, the useful data rate increases with modulation order, coding rate, number of spatial layers, and resource allocation. The central quantity in this trade-off is the effective spectral efficiency,

$$\eta_{eff} = R_c \log_2(M)\, L\, (1 - OH_{DMRS} - OH_{CP}) \tag{101}$$

where Rc is the coding rate, M is the modulation order, L is the number of spatial layers as in equation (26), and the overhead terms represent the DM-RS and cyclic-prefix overheads of the resource grid. Control-channel overhead does not appear because the baseline models the PDSCH data channel only, as fixed in Section 2.4. Equation (101) captures the main engineering tension of the whole project: increasing M and L raises the peak spectral efficiency, but the gain is realized only if the channel estimator, the equalizer, and the LDPC decoder can support the resulting reliability requirement, which is exactly what the BLER results of Chapter 11 measure.

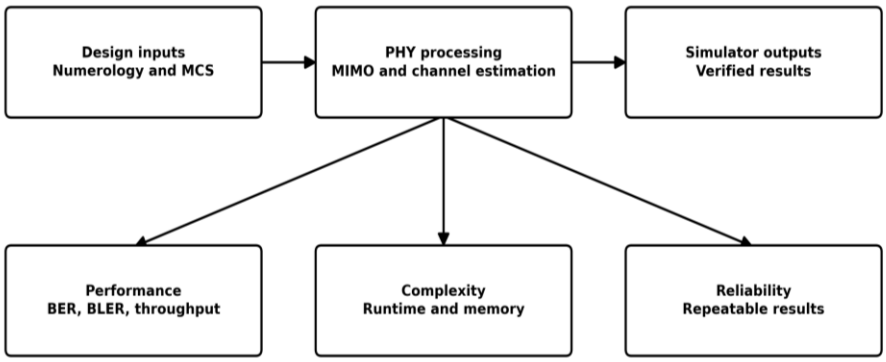


Figure 38. Engineering trade-off map for the PDSCH MIMO-OFDM simulator.

Figure 38 summarizes the trade-off structure used in this project. The design inputs, numerology and MCS, feed the PHY processing stage, MIMO and channel estimation, whose outputs are judged against three competing objectives: performance in BER, BLER, and throughput, complexity in runtime and memory, and reliability in the form of repeatable, verifiable results. The selected design point must be accurate enough for 5G NR PHY analysis,

simple enough to implement in both MATLAB and Python, and repeatable enough to support the CSV-based result acceptance of Section 11.1.

12.2 Waveform, Modulation, and OFDM Trade-offs

The subcarrier spacing sets the symbol-time quantities

$$T_u = 1 / \Delta f \tag{102}$$

$$T_{sym} = T_u + T_{CP} \tag{103}$$

and the data capacity of the grid is

$$N_{RE,data} = N_{RB}\, N_{SC,RB}\, N_{sym,data} - N_{RE,DMRS} \tag{104}$$

At the 30 kHz baseline, equation (102) gives a useful symbol of 33.3 μs, long against the TDL delay spreads of Section 6.4, so the normal cyclic prefix absorbs the multipath without excessive overhead; the numerology is kept fixed so receiver behavior can be compared across channels without mixing numerology effects, which is also what kept the two implementations directly comparable during cross-verification. Each modulation order carries

$$b_{RE} = log_2 M \tag{105}$$

bits per resource element but shrinks the constellation distance, so the higher orders demand a higher post-equalization SINR, and the effective throughput

$$T_{eff}(SNR) = T_{peak}\,(\,1 - BLER(SNR)\,) \tag{106}$$

with BLER per equation (42) explains why the highest order is not always the best choice: the steep uncoded throughput threshold measured in Section 11.8 is exactly this trade-off with the channel code removed, and channel coding is what turns the threshold into a graded waterfall.

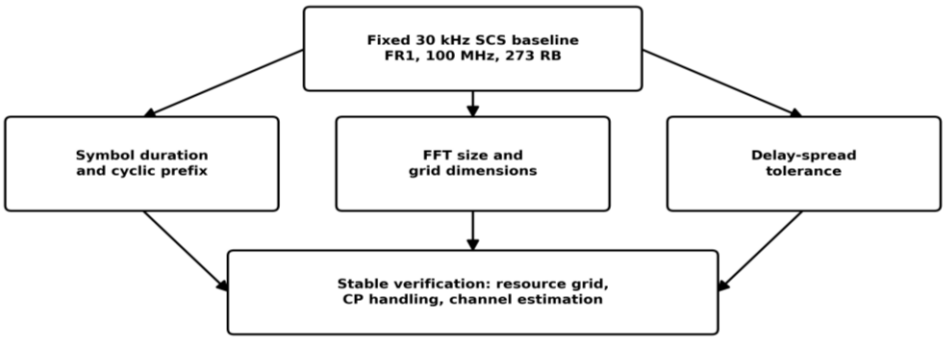


Figure 39. Numerology and OFDM design trade-off.

The numerology trade-off is shown in Figure 28. The fixed 30 kHz baseline determines the symbol duration and cyclic prefix, the FFT size and grid dimensions, and the delay-spread

tolerance at the same time, so freezing it keeps the resource grid, the cyclic-prefix handling, and the channel-estimation structure stable.

12.3 MIMO, Estimation, and Simulation-Cost Trade-offs

More layers raise the multiplexing gain but also the sensitivity to channel rank and noise enhancement. The ZF output noise for layer l,

$$\sigma_{ZF,l}^2 = \sigma_n^2 \, [\, (\, \hat{G}^H \, \hat{G} \,)^{-1} \,]_{ll} \tag{107}$$

grows with the diagonal of the inverted Gram matrix and diverges near rank deficiency, while MMSE regularizes exactly this inverse with the $\sigma n^2 I$ term of equation (39); Section 11.5 confirms MMSE below ZF over the whole measured range at full spatial load. The DM-RS overhead

$$OH_{DMRS} = N_{RE,DMRS} \, / \, N_{RE,total} \tag{108}$$

buys estimation accuracy measured by the NMSE of equation (78): the executed campaign shows the unsmoothed LS estimate tracking the pilot noise floor, and the measured price of that estimate is the MMSE-LS curve crossing above ideal-CSI ZF in Figure 24. The statistical uncertainty of a BER estimate,

$$SE_{BER} = \sqrt{(\, BER \, (\, 1 - BER \,) \, / \, N_{bit,total} \,)} \tag{109}$$

sets the runtime floor: the forty frames per SNR point of the executed campaign bound the resolvable error and block-error rates, which is why the 25 dB entries of Tables 11 and 12 carry the widest Monte Carlo spread. The layer-domain capacity,

$$C_L[k] = \log_2 \det(\, I_L + (\rho / L) \, G^H[k] \, G[k] \,) \tag{110}$$

complements the antenna-domain capacity of equation (45) by using the effective layer channel of equation (34); it is the capacity reference tabulated for the TDL campaign in Tables 11 and 12, and it is never interpreted as achieved coded throughput. The selected baseline is deliberately not the most complex simulator that could have been written: it is a controlled, verifiable implementation whose every result is traceable, and whose extension path is separated from the verified core.

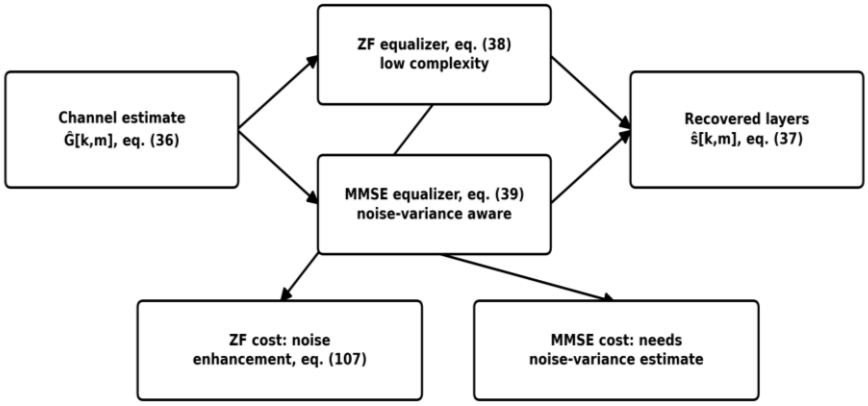


Figure 40. MIMO layering and equalizer trade-off.

Figure 40 presents the equalizer design trade-off. The ZF branch offers full interference removal at the cost of the noise-enhancement risk of equation (107), while the MMSE branch is noise-variance aware and robust at low SNR at the cost of requiring a noise-variance estimate. Keeping ZF as the low-complexity baseline and MMSE as the robust receiver matches the standard linear-receiver options for MIMO-OFDM link-level studies and gives the project a measurable design comparison rather than a single fixed choice.

Chapter 13. Conclusions and Future Work

This chapter concludes the report. It summarizes what the project built and what the executed campaigns measured, states the engineering contributions and the scope boundaries, and defines the future extensions of the simulator.

13.1 Technical Conclusions

This project delivered a complete engineering framework for a 3GPP 5G NR PDSCH-based Massive MIMO-OFDM physical-layer link-level simulator: requirements, architecture, mathematical model, channel models, algorithms, two independent implementations, a verification methodology, executed and cross-verified results, and the trade-off analysis, in one traceable structure. The same processing chain runs unchanged over all channel models, which is what makes the measured comparisons of Chapter 11 fair.

The executed results support four conclusions. First, the implemented chain is numerically correct: the uncoded BER lies on the closed-form AWGN references of equations (95) and (96) across five decades for QPSK and 16-QAM, and on the flat-Rayleigh expression of equation (97) with the diversity-one slope. Second, estimation quality is measurable and it propagates: the LS NMSE tracks the pilot noise floor from 0.49 at 0 dB to 3.9×10^{-3} at 25 dB, exactly as equation (66) predicts, and the cost of using that estimate appears directly as the MMSE-LS curve crossing above ideal-CSI ZF near 10 dB in Figure 24. Third, the equalizer derivations hold in measurement: MMSE stays below ZF over the full measured range at full spatial load, as equations (38), (39), and (107) predict. Fourth, the massive array delivers its promised gain: the SVD-precoded 64×8 configuration runs error-free above 5 dB, zero errors in 1.7×10^{6} bits per point, while the unprecoded 4×4 system remains above 4×10^{-2} at 20 dB at the same total power and layer count, a separation of at least 19 dB at the 10^{-2} error rate, reproduced independently by both implementations.

13.2 Contributions

The main contribution is a verified, report-and-code simulator for 5G NR PDSCH link-level evaluation. The work organizes the PHY chain into implementable modules, defines how each module is verified, and closes the loop by executing the campaigns and accepting the results under fixed criteria: the configuration is locked by equation (83), the model by the unit-power and dimension checks of equations (14) and (90), the algorithms by the unit tests of Section 10.2, and the results by the executed campaigns with stored CSV files. The cross-verification record of Table 13 closes the dual-implementation claim: thirty metric comparisons across six SNR points, all inside the tolerances of Table 8, with worst-case deviations of 0.236 decades in BER and 0.175 absolute in BLER at the statistically coarsest point. Every number in Chapter 11 traces to a stored file and a locked configuration, which avoids the common weakness of portfolio reports where theory is disconnected from implementation and measurement.

13.3 Technical Scope and Boundaries

The simulator is a physical-layer link-level project: it does not model MAC scheduling, RLC, PDCP, RRC signaling, mobility, HARQ protocol timing, or core-network procedures, and it does not claim 3GPP conformance testing, as stated openly in Chapter 10. RF impairments, phase noise, IQ imbalance, carrier and sampling offsets, power-amplifier nonlinearity, calibration error, and receiver quantization, are separated from the verified core per Section 2.4, so an error in the verified baseband can never hide behind an impairment model. One item remains open by design: the LDPC coded-mode campaign on the MATLAB 5G Toolbox path of Section 8.3, whose BLER and throughput curves enter Chapter 11 only after execution, per the provenance rule of equation (99). The MATLAB-Python cross-verification, previously the other open item, is closed by the executed record of Section 11.7.

13.4 Future Work

The immediate task is the coded-mode campaign: execute the LDPC-coded PDSCH runs through the same acceptance pipeline and add the resulting BLER and throughput waterfalls to Chapter 11, turning the threshold behavior of the uncoded transport block into the graded coding-gain curves that equation (106) anticipates. After that baseline closes, extensions follow in order of verification cost: additional numerologies, detailed LDPC base-graph and rate-matching behavior, LMMSE smoothing and time-frequency interpolation on the executed DM-RS pattern, expanded DM-RS and PTRS configurations, HARQ combining, Doppler-dependent TDL channels, antenna correlation, beamforming codebooks, and link adaptation, each added only while the corresponding baseline result remains reproducible under the regression protocol of Section 10.5.

Two research directions complete the roadmap. RF-aware PHY simulation adds phase noise, IQ imbalance, and power-amplifier nonlinearity with EVM and ACLR-related transmitter-quality metrics, calibration error, and receiver quantization, connecting the simulator to RF system specification and RF/PHY algorithm work, especially combined with the beamforming analysis that the massive MIMO results of Chapter 11 already support. Receiver research uses the platform to compare AI/ML-assisted channel estimation and detection against the measured LS, MMSE, and ZF baselines, and to validate the chain against reference-tool outputs such as the MATLAB 5G Toolbox link examples; because every baseline result is stored with full provenance, any such comparison starts from a known, reproducible reference

Chapter 14. References

[1] 3GPP TS 38.300, “NR; NR and NG-RAN Overall Description; Stage-2.”

[2] 3GPP TS 38.211, “NR; Physical channels and modulation.”

[3] 3GPP TS 38.212, “NR; Multiplexing and channel coding.”

[4] 3GPP TS 38.214, “NR; Physical layer procedures for data.”

[5] 3GPP TR 38.901, “Study on channel model for frequencies from 0.5 to 100 GHz.”

[6] 3GPP TS 38.101-1, “NR; User Equipment (UE) radio transmission and reception; Part 1: Range 1 Standalone.”

[7] E. Telatar, “Capacity of multi-antenna Gaussian channels,” European Transactions on Telecommunications, vol. 10, no. 6, pp. 585-595, 1999.

[8] J. G. Proakis and M. Salehi, Digital Communications, 5th ed., McGraw-Hill, 2008.

[9] A. Goldsmith, Wireless Communications, Cambridge University Press, 2005.

[10] D. Tse and P. Viswanath, Fundamentals of Wireless Communication, Cambridge University Press, 2005.

[11] E. Dahlman, S. Parkvall, and J. Skold, 5G NR: The Next Generation Wireless Access Technology, 2nd ed., Academic Press, 2020.

[12] MathWorks, 5G Toolbox: User's Guide, Natick, MA, USA.

[13] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357-362, 2020.